

Evaluating C/C++ Vulnerability Detectability of Query-Based Static Application Security Testing Tools

Zongjie Li^{1b}, Graduate Student Member, IEEE, Zhibo Liu^{1b}, Wai Kin Wong^{1b}, Pingchuan Ma^{1b},
and Shuai Wang^{1b}, Member, IEEE

Abstract—In recent years, *query-based static application security testing* (Q-SAST) tools such as CodeQL have gained popularity due to their ability to codify vulnerability knowledge into SQL-like queries and search for vulnerabilities in the database derived from the software. The industry has made considerable progress in building Q-SAST tools, facilitating their integration into the continuous integration (CI) pipeline, and sustaining an active community. However, we do not have a systematic understanding of their vulnerability detection capability in comparison to conventional SAST tools. We conduct the first in-depth study of Q-SAST to demystify their C/C++ vulnerability detectability. Our study is conducted from three complementary aspects. We first use a synthetic CWE test suite and a real-world CVE test suite, totaling almost 30 K programs with known CWE/CVE, to assess popular (commercial) Q-SAST and industry-leading SAST (requiring no queries). Then, we gather defect-fixing pull requests (PRs) since the release dates of three popular Q-SAST tools, characterizing historically-fixed defects and comparing them to pitfalls exposed in our CWE/CVE study. To enhance vulnerability detection, we design SAST-MT, a metamorphic testing framework to detect false positives (FPs) and false negatives (FNs) of Q-SAST. Findings of SAST-MT can be used to easily expose the root causes of Q-SAST’s FPs and FN. We summarize lessons from our study that can benefit both users and developers of Q-SAST.

Index Terms—SAST, static analysis, vulnerability detection.

I. INTRODUCTION

DISCOVERING vulnerabilities is a classic yet challenging problem. Constantly analyzing large amounts of source code, in particular, has been a major focus of software engineering research. Among the various paradigms of security analysis tools [17], [71], [140], static application security testing (SAST) tools [69], [91] have become one of the mainstream due to their high flexibility for analyzing software without running it and their typically greater comprehensiveness when compared to dynamic application security testing (DAST) tools [16], [140].

Manuscript received 16 September 2022; revised 20 October 2023; accepted 12 November 2023. Date of publication 16 January 2024; date of current version 4 September 2024. This work was supported in part by a RGC ECS grant under the contract 26206520. (Zhibo Liu and Wai Kin Wong equally contributed for this work.) (Corresponding author: Shuai Wang.)

The authors are with the Department of Computer Science and Engineering, HKUST, Clear Water Bay, Hong Kong (e-mail: zligo@cse.ust.hk; zliudc@cse.ust.hk; wkwongal@cse.ust.hk; pmaab@cse.ust.hk; shuaiw@cse.ust.hk).

Digital Object Identifier 10.1109/TDSC.2024.3354789

Recent years have seen an increase in the popularity of *query-based* SAST tools (referred to as **Q-SAST**). De facto Q-SAST tools share a key design concept by codifying vulnerability patterns as SQL-like queries. Codified vulnerability patterns ease knowledge sharing and reusing across different entities and analyzing different software. Accordingly, the software is treated as data [137], with programs parsed into hierarchical representations and stored usually in databases. Q-SAST tools primarily focus on parsing software into rich collections of query-friendly representations, and encouraging the community to contribute queries to continuously and timely combat vulnerabilities. Visible Q-SAST tools sustain active communities, and bounty programs [2] are coordinated to encourage query contribution and enhancement. The industry and the community have made tremendous progress in creating and promoting Q-SAST tools. Today, among the top-10 most installed/stared Github security apps that can detect C/C++ vulnerabilities [15], five can be categorized as Q-SAST. Industry-leading Q-SAST tools such as CodeQL [72] have been integrated into GitHub [31] and are also extensively used to check production code [29]. It is disclosed that Microsoft requires all driver submissions must be scanned with CodeQL [28]. SonarQube [123] (a well-known commercial tool) enterprise licenses can cost several thousand dollars. Despite the community’s widespread and overall enthusiasm toward Q-SAST, we demand an impartial and in-depth assessment of their “vulnerability detectability;” in this paper, vulnerability detectability refers to the detection capabilities and effectiveness of Q-SAST tools. Given that Q-SAST tools are primarily used to discover security problems, analysis mistakes may have a major impact on the software’s trustworthiness.

While it may seem self-evident that static analysis tools find it difficult to precisely detect all flaws in real-world software, we aim to quantify practically “how far” Q-SAST has come at this point. Particularly, we believe that it is critical to demystify the following: ① whether de facto Q-SAST can outperform conventional SAST without queries (studied in Sections III-B and V), ② Q-SAST’s genuine capability for detecting vulnerabilities in C/C++ code (studied in Section V), ③ Q-SAST’s common yet under-documented pitfalls that users may encounter (discussed in Sections V and VI), and ④ potential enhancement on Q-SAST (Sections VII and VIII). We explore each aspect in detail, present lessons along with the study, and summarize key take-away messages in Section VIII.

This paper systematically demystifies the capability of Q-SAST, which to our knowledge, has yet to be assessed comprehensively. We examined 19 popular (commercial) SAST and focused our in-depth study on four (commercial) Q-SAST. We also examine three industry-leading SAST that solely features built-in vulnerability patterns (we refer to such tools as **NQ-SAST** in this paper). Following that, we conduct a three-step study. ① We first use a large-scale synthetic CWE test suite (Juliet [105]) and a real-world CVE dataset [103], including about 30 K CWE/CVE programs, to assess the vulnerability detection capabilities of those seven Q-SAST/NQ-SAST. We manually inspect the false positives (FPs) and false negatives (FNs) of them to study the root causes and summarize the findings. ② We next examined over 1.2 K *defect-fixing* pull requests (PRs) from the GitHub repositories of three popular Q-SAST tools, which represent changes proposed by contributing developers to fix identified defects or issues. After reviewing and categorizing the root causes addressed in merged PRs, we compare them with the remaining pitfalls exposed in ① to gain further insights into improving the quality of tools for the community. ③ We also develop a proof-of-concept framework, SAST-MT, to launch metamorphic testing (MT) [53] to test CodeQL and expose its FPs/FNs. SAST-MT performs semantics-preserving mutations to marginally and iteratively mutate a seed; localizing root causes of FP/FN becomes much easier by automatically diffing two adjacently mutated versions (where the latter triggers a FP/FN). After 100 hours of testing, SAST-MT exposed 17 FPs and 228 FNs in the latest version of CodeQL. In sum, we make the following contributions:

- We, for the first time, comprehensively study de facto Q-SAST's vulnerability detectability for C/C++ code. We select seven industry-leading (commercial) Q-SAST/NQ-SAST. We use synthetic CWE programs and real-world software with known CVEs to assess them. We also analyze their defect-fixing PRs to compare characteristics of historically-fixed defects and the remaining pitfalls.
- Assessing (closed-source) static analysis tools and comprehending their FPs/FNs is challenging; our manual study took in a total of 540 man-hours, covering about 30 K programs containing CWE/CVE vulnerabilities and 1.2 K PRs gathered since the release dates of three prominent Q-SAST. We analyzed 8.7 K FPs and 22.7 K FNs of two Q-SAST to summarize root causes. We present lessons on their pitfalls, usage, and improvement.
- Based on our findings, we present SAST-MT, a proof-of-concept MT framework for CodeQL. SAST-MT features incremental and iterative semantics-preserving mutations to automatically expose FPs/FNs of CodeQL. In this MT setting, the root cause analysis of FPs/FNs can be easily and automatically done by diffing two adjacently mutated versions, thus largely easing the procedure of enhancing CodeQL's vulnerability detection capability without tedious manual efforts.

Artifact Availability: We maintain a website to host all research artifacts, including 1) FP/FN root cause distributions of all CWE cases (except CWE121 and CWE190 which have been discussed in Section V-A), 2) details of CodeQL-detected

real-world CVEs, 3) PR analysis raw data, and 4) code and findings of SAST-MT at the following link:

<https://sites.google.com/view/qsast-survey>

We will maintain this website to benefit follow-up usage and improvement of Q-SAST. Moreover, the documentation of our SAST-MT tool, as well as the source code for the experimental framework, is available for download and permanently hosted by Zenodo at the following link: <https://zenodo.org/record/7953985>.

II. BACKGROUND

We motivate this research by first distinguishing bug detection and vulnerability detection in Section II-A. We then introduce the technical pipeline of Q-SAST tools in Section II-B, and highlight the research motivation in Section II-C.

A. Bug Detection Versus Vulnerability Detection

A software “bug” refers to an error that causes incorrect or unexpected functionality, which can occur during design, development, or deployment [124]. As defined in [44], a “vulnerability” refers to an internal fault that enables an external fault to harm the system, the exploitation of which may enable adverse impacts (e.g., memory overwrite). We believe detecting vulnerability has distinct requirements than detecting bugs for the following reasons:

Vulnerability Diversity: Vulnerabilities often manifest diverse patterns; they are not just bugs. Many unsafe designs or implementation patterns may result in vulnerabilities. Design flaws such as inadequate crypto algorithms, lack of security features (e.g., authentication), wrong threat models, or even side channels can result in exploits. Bugs, hardware problems, and non-compliance with security standards are examples of implementation-level flaws. The community maintains a list of common weakness enumeration (CWE), including 924 distinct vulnerability patterns [11]. Also, vulnerabilities uncovered in real-world software may be granted a CVE ID (each CVE should be paired with its relevant CWE). The CVE database [13] has archived over 175 K CVEs as of May 2022.

Vulnerability Emergingness: While bugs are often well summarized into limited types, new vulnerabilities are constantly found. Since 2017, about 10K CVEs have been disclosed annually [61], with the mean time to remediate (MTTR) for each case taking nearly two months [1].

B. Q-SAST Tools

Fig. 1 depicts the workflow of using modern Q-SAST tools for vulnerability detection. The pipeline is typically formed by three components [9], [113], [128]: the query language (and its compilation toolchain), the software-derived database, and the search engine that uses queries to search the database.

Software Parsing and Analysis: Q-SAST parses software into analysis-friendly representations, which are usually maintained in databases. Q-SAST tools often recover multi-layered hierarchical representations of software. For instance, CodeQL

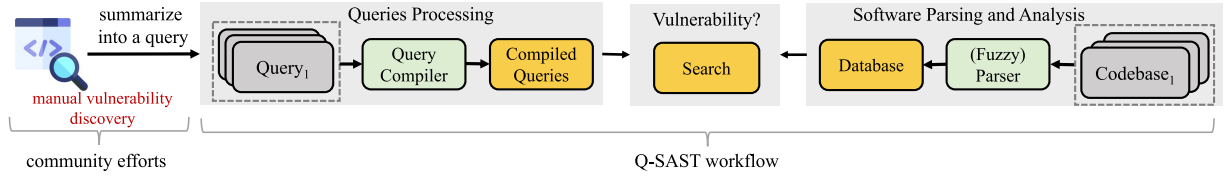


Fig. 1. Technical pipeline of modern Q-SAST tools.

```

1. class ByteOrderTrans extends Expr {
2.   ByteOrderTrans() { this.Name().Match("ntoh") }
3. }
4. class NetToMemLen extends TaintTracking {
5.   predicate isSource(Node source) {source.asExpr() instanceof ByteOrderTrans}
6.   predicate isSink(Node sink) {
7.     exists(FuncCall fc | fc.Name().Match("memcpy") and fc.Arg(2) = sink.Expr())
8.   }
9. }
10. from Expr ntoh, Expr sizeArg, NetToMemLen config
11. where config.hasFlow(ntoh, sizeArg) select ntoh.Loc(), sizeArg

```

Fig. 2. Query detecting buffer overflow by checking untrusted network function output propagating to memcpy.

recovers abstract syntax trees (ASTs), data flow graphs (DFGs), and control flow graphs (CFGs), and dump them into databases. Static analyses like taint analysis, range analysis, and semantic code search [43] may be performed to enrich ASTs/DFGs/CFGs with auxiliary information. Taint information obtained by Q-SAST like Joern [128] shows high effectiveness in detecting buffer overflow and SQL injection [43], [121].

Fuzzy Parsing: Q-SAST tools [113], [141] may feature *fuzzy parsing*, allowing code fragments (with incomplete context and dependencies) to be parsed and analyzed without building the software beforehand. This might be helpful in the continuous integration (CI) scenarios, since fuzzy parsing enables the analysis of code snippets without building the entire software first (which might be infeasible).

Query Language: Vulnerabilities are codified in query languages. For instance, CodeQL query is written in a Datalog variant; each CodeQL query is compiled into Datalog and then used to query relational databases extracted from software. Queries can be *specific* to reduce FPs when analyzing large codebases. For instance, users can check use-after-free on a suspicious pointer named `AudioHandler` in Chrome [26]. This is hardly possible for NQ-SAST tools because they are often not configurable toward specific functions/variables/APIs. Considering the query in Fig. 2, which detects the buffer overflow. It checks if the `ntoh` function's output, as network packages (line 2), can propagate to the third parameter of `memcpy` (line 7). `NetToMemLen`, inherited from the `TaintTracking` class of CodeQL, specifies a predicate as the taint source (line 5) and another as the taint sink (lines 6–8). Lines 10–11 form a SQL-like query: `where` specifies vulnerability pattern (in terms of taint flow), and `select` specifies the query output. Non-trivial outputs illustrate that the size of the copied memory chunk is controllable by attackers, a severe flaw.

Community-Empowered Query Creation/Enhancement: The community can contribute to Q-SAST by creating or enhancing

queries [2], [12], [22], [24]. For instance, CodeQL rewards the community for contributing queries via the bounty program [2]: contributors submit queries via PRs to the CodeQL GitHub repo and receive bounties when the queries are accepted. Via the bounty program, the community is believed to actively create new queries and improve existing ones. The community can make a query more general, adaptable to different codebases, or more precise, to reduce FPs. This way, a new query can be widely used to combat vulnerability breaches.

Vulnerability Search: Given a query derived from codified vulnerability patterns, Q-SAST tools search against databases formed by software. Q-SAST tools frequently represent a query into a taint-style description [137], which uses a 3-tuple to represent sources controlled by attackers, security sensitive sinks (e.g., memory copies), and sanitization checks (`if` statements). Then, multiple SQL-like queries are derived to check over databases [23] and decide if information flow from source to sink exists, whereas sanitization checks on the flow are absent. Not all vulnerabilities can be encoded into taint-style descriptions. Semgrep [113] may also accept ASTs converted from queries, and search against the target program that is also in the form of AST.

C. Significance & Study Scope

Q-SAST Significance: Q-SAST tools are crucial for identifying and fixing software vulnerabilities. Their ability to identify CVEs in real-world software [10] has generated enthusiasm across both the security community and industry. Industry-leading Q-SAST tools such as CodeQL [72] have been integrated into GitHub [31] and also extensively used to check production code in Microsoft [29], including all driver submissions [28]. Improvements are constantly undertaken by the community and the Q-SAST developers [2], [12], e.g., Github security team enhances the vulnerability detectability of CodeQL toward less-common software with machine learning [18].

Q-SAST Advantage Over NQ-SAST: We have noticed recent research studies [91], [96] that indicate comparable or even better accuracy of Q-SAST over (commercial) NQ-SAST regarding CVE detection. The community also illustrates the enthusiasm for the “community-powered security analysis” concept advocated by Q-SAST [2], [2], [12], [22], [24], [84]. Typically for NQ-SAST, only their developers/maintainers write bug/vulnerability patterns. For Q-SAST, the community is often rewarded (e.g., via bounty programs) to constantly create and polish queries to detect vulnerabilities. This enables timely combat with diverse and emerging vulnerabilities in real-life scenarios. Fig. 3 illustrates how the community is actively

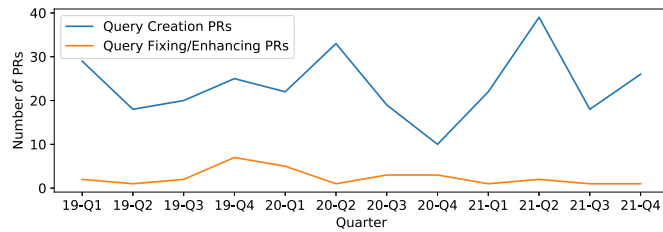


Fig. 3. CodeQL query creation PRs and query bug fixing PRs issued by the community.

contributing queries to CodeQL via pull requests (PRs).¹ As shown, the community has been actively creating or enhancing/fixing queries since CodeQL became free available in Q4 2019. It is seen that the amount of query enhancing/fixing PRs in each quarter (the orange line in Fig. 3) is declining. In comparison, many new queries are constantly accepted by CodeQL every quarter. We interpret that the quality of queries provided by the community are steadily increasing.

III. STUDY SETUP

A. Study Scope

We discuss the study scope from two aspects. First, static bug detection has been studied for decades. Nonetheless, vulnerability detection necessitates high extensibility to timely combat diverse and emerging vulnerabilities. It is thus questionable whether “bug detectors” can cope with security breaches. Therefore, unlike the many articles examining bug detection of static tools [116], [132], [133], this study primarily focuses on evaluating *vulnerability detection*, particularly CWE/CVE cases. And as a practical and demanding objective, we focus on C/C++ vulnerabilities, given their pervasiveness in real-world scenarios.

Second, unlike most existing efforts on the vulnerability detectability of NQ-SAST (reviewed in Section IX), we focus on Q-SAST, which has manifested its great significance in recent years. Our study roots from the following key question:

“Is Q-SAST really outperforming NQ-SAST in detecting C/C++ vulnerabilities?”

This is a demanding yet unresolved question. We carefully design empirical studies from several aspects and involve a large volume of human efforts to explore this question. By allowing vulnerability patterns to be codified into queries and published online, Q-SAST may better combine community power to combat vulnerabilities. This paper empirically confirms this intuition to a degree. Nevertheless, despite the non-negligible community enthusiasm, we highlight under-documented characteristics and even pitfalls of Q-SAST that need improvement and users’ attention. Overall, though static analysis is fundamentally undecidable [92], we empirically reveal “how far we have come,” delineating the capability of Q-SAST. We also outline lessons and guidance for Q-SAST users and their developers.

¹The standard approach (including the “bounty programs” [2]) for the community to contribute to CodeQL is via PRs.

B. Tool Selection

This section presents the procedure of selecting the proper Q-SAST for the study. We also select well-performing NQ-SAST for comparison. We first report the seven selected tools in Table I. We now detail the selection criteria. Each Q-SAST offers queries to detect specific C/C++ vulnerabilities, and we report the C/C++ CWE coverage in Table I. To our best knowledge, there is no document comprehensively enumerating the total number of CWEs that may exist in the C/C++ code. However, MITRE [101] identifies 86 CWEs specifically found in C++ programs.² It should be noted that this number only counts CWE issues specifically found in C++ programs, and it does not include general software vulnerabilities that can appear in any language. Therefore, MITRE’s number may be less than the coverage numbers shown in Table I, as those numbers include both general CWEs as well as CWEs specifically found in C/C++.

We now explain the selection procedure of these seven tools. At this step, we first reviewed prior studies on SAST tools [52], [68], [70], [83], [95], [98], [122], [129], [133] as well as non-academic sources [42], [130]. We studied well-established SAST tools, including Clang, Facebook Infer [67], NASA IKOS [49], [104], Coverity [126], Google Project Zero weggli [141], Frama-C [60], CPPCheck [66], CodeQL, and Joern. We assessed all these well-known tools, and eventually decided to keep CodeQL, Joern, Clang, and Coverity for our in-depth study. In short, we find that some visible bug detectors, including Infer, IKOS, Frama-C, and CPPCheck, can only detect very limited CWE cases. weggli seems less mature than the others. As shown in a recent study [96], CodeQL, Clang, and Joern are able to detect 2.45 times more real-world CVEs on average than other SAST tools such as Infer, CPPCheck, and IKOS. Due to their superior performance, we add them to our target analysis tools, and details of them are as follows:

CodeQL is an industry-leading closed-source Q-SAST tool. It is marketed that over 700 K developers have used CodeQL to analyze over 135 K open-source projects, with all results available at lgtm.com. We introduced the software parsing and queries of CodeQL in Section II-B. As in Table I, CodeQL possesses 560 queries for C/C++. Typically several queries correspond to different variants of one CWE, and all these available queries cover 146 C/C++ CWEs [3].

Joern, as seen in Table I, allows fuzzy parsing. It parses software into semantic code property graphs (SCPGs), which are stored in OverflowDB [23]. SCPGs subsume program structure and semantics information [137]. Joern performs taint analysis to enrich the SCPGs. Joern provides only 27 queries. It, however, provides a translator to convert CodeQL queries into its own queries.

Clang is a free NQ-SAST developed by the LLVM community. **Coverity**, developed by Synopsys, is a closed-source commercial NQ-SAST. We follow a previous study [79] toward

²To clarify a confusion: this study does not merely consider CWEs *specific* to C/C++. Rather, we consider CWEs that may appear in C/C++ code, which subsume CWEs that are unique to C/C++ and CWEs that exist generally in software written in different languages.

TABLE I
SAST SELECTED IN THIS STUDY

Category	Tool Name	Version	Fuzzy Parsing	Accept User Queries	#C/C++ Queries Available	#C/C++ CWE Coverage
Q-SAST	CodeQL [72]	2.6.2	No	Yes	560	146 [33]
	SonarQube [123]	9.3.0.51899	No	Yes	883	57 [34]
	Joern [128]	1.1.409	Yes	Yes	27+560(?) ^b	≈ 146 ^c
	Semgrep [113]	0.79.0	Yes	Yes	86	NA ^d
NQ-SAST	Mend [135]	22.2.2.3	Yes	No	3,268	148
	Clang ^a [57]	11.0.0	No	No	NA	NA
	Coverity [126]	2021.12	No	No	NA	101 [127]

(a) Clang represents Clang Static Analyzer [57].

(b) Besides 27 default queries, Joern can (partially) use CodeQL's queries.

(c) Its default 27 queries have limited CWE coverage. However, due to (b), Joern may have close to 146 C/C++ CWE coverage.

(d) Semgrep's queries are not annotated for CWEs, and it is hard to map its queries into CWEs on our end.

Coverity to setup its cloud-based scanner, Coverity Scan, in our study. As disclosed by Coverity [127], 101 C/C++ CWEs are detectable. Our preliminary study shows that Clang and Coverity are of high engineering quality, and they notably outperform other NQ-SAST tools in detecting CWEs.

There are many Q-SAST and NQ-SAST on the market, and at this step, we examined 19 popular (commercial) SAST in total. Additionally, we consider popular tools on the market that have recently attracted attention from the security analysis community but have not yet been thoroughly studied. We studied top-10 most installed/stared Github security apps [15], where five tools, CodeQL, Semgrep, SonarQube, Mend, and GuardRails, can detect C/C++ vulnerabilities. We exclude GuardRails since it is built on Semgrep. As CodeQL was already introduced, we focus here on the remaining three tools.

Semgrep is an open-source Q-SAST tool. Its query language is regular expression-like. Semgrep is stated mainly for speedy analysis, since it does not perform expensive static reasoning. Semgrep offers 86 queries to help detect C/C++ vulnerabilities [74]. We find its manuals for each query are less clear, and the C/C++ CWE coverage of these queries is unclear.

SonarQube [123] is a popular closed-source commercial Q-SAST. SonarQube has 883 queries to check code smell, bug, and vulnerabilities, covering 57 C/C++ CWEs. Users can incorporate their own queries by editing its templates. According to its document, SonarQube expects zero FP for bug detection and over 80% TPs for vulnerability detection (albeit their test suites are not mentioned). We purchase its C/C++ commercial license.

Mend [135], previously known as “WhiteSource”, is a closed-source commercial NQ-SAST. Mend seems particularly focus on finding security flaws. Mend does not allow users to provide queries. However, it is stated to detect 2,368 vulnerability patterns where most of them have a CVE ID [19]. We crawled all its supported CVEs from [19] and found that they can be mapped into 148 unique C/C++ CWEs.

Q-SAST and NQ-SAST tools listed in Table I, to our best knowledge and empirical exploration, are highly representative, if not the best, SAST tools available to the public for detecting C/C++ vulnerabilities.

Our study reveals some renowned NQ-SAST tools detect limited C/C++ security flaws. These tools do not accept user-provided queries. Though they often feature advanced static

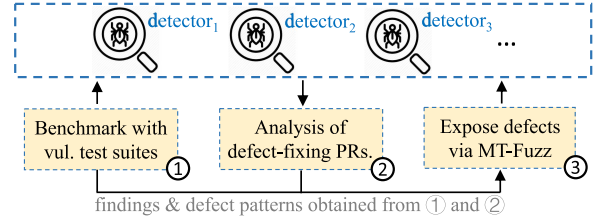


Fig. 4. Overview of step-three study methodology.

analysis (e.g., abstract interpretation [58]), it may be accurate to believe they mainly target common bugs instead of diverse vulnerabilities in the wild; see future comparison between Q-SAST and NQ-SAST in Section V. Also, as shown in Table I, our selected seven Q-SAST/NQ-SAST cannot cover full C/C++ CWEs. We believe the main reason is that some CWEs to date may be outdated and rarely seen in real-world C/C++ software, and it would take quite an effort for the community to complete the full sets of C/C++ CWEs.

IV. A THREE-STEP STUDY PIPELINE

Conceptually, as illustrated in Fig. 4, our study pipeline comprises three steps: ① We first benchmark those SAST tools selected in Section III-B with CVE/CWE test suites, manually inspect and characterize all analysis inaccuracy. ② Then, we examine their defect-fixing PRs, characterizing root causes of historically-fixed detects. We compare their remaining pitfalls exposed in ① with fixed defects, revealing under-documented challenges in developing high-quality Q-SAST tools, and potential design tradeoffs/recommendations that can be considered by Q-SAST developers. ③ Furthermore, we provide a proof-of-concept MT tool to uncover incorrect analysis outputs of CodeQL, thereby amplifying its vulnerability detection effectiveness. The three progressive steps outlined above offer insight into the capabilities of Q-SAST tools and correspond with key contributions discussed in Section I. Details of each step are provided below, following definitions of metrics used in our analysis.

A. Metrics

Before introducing the detailed description of each step, we first define the metrics used in this research as follows:

TABLE II
STATISTICS OF C/C++ PROGRAMS USED

	JULIET		REAL
#programs	30,950	#software	51
#CWE	12	#CVE	136
LOC per program	104.1	LOC per software	341,559.9

We use cloc [62] to count line of code (LOC).

TABLE III
DEFECT-FIXING PR ANALYSIS

Tool	#PR	#AF	#FP	#FN	#(FP+FN)
CodeQL	206	0	43	0	1
Joern	312	7	0	6	7
Semgrep	770	38	19	64	15

False Positives (FPs): Each FP indicates that a safe code snippet is flagged as “vulnerable.” FPs are generally not avoidable, and users will often need to post-check findings of SAST tools to distinguish FPs and true positive (TP) findings.

False Negatives (FNs): Each FN denotes a neglected TP. FNs are often more severe than FPs in security, denoting potential attack vectors remaining in the codebase after SAST scans.

Analysis Failures (AFs): We record crashes or exceptions that terminate SAST tools. These usually indicate parsing failures in queries or input software.

To demystify C/C++ vulnerability detection, we primarily concern FPs/FNs. Developers may use fuzzing or regression testing to easily find AFs before release. Regardless, we record all AFs we encounter during the study.

B. Step ①: Benchmarking With CVE/CWE

To provide a comprehensive and fair assessment, we use synthetic CWE and real-world CVE test suites to benchmark Q-SAST/NQ-SAST. Table II summarizes the statistics. Inspired by previous work on benchmarking static analysis tools [107], we choose JULIET [105], which is one of the most well-known benchmarks in the Software Assurance Metrics and Tool Evaluation (SAMATE) project from NIST [106], as the synthetic benchmark. JULIET is a large test suite, composing C/C++ synthetic programs of 118 different CWEs. Each CWE test program has several “good” variants with no vulnerability and a “bad” variant with a vulnerability. Hence, tools can be evaluated regarding flaws found in bad variants but not in good variants. We cannot evaluate all JULIET test cases since we need to *manually* analyze every FP/FN (see Section V-A). We first pick all “2021 Top 25 Most Dangerous Software Weaknesses”(CWE Top 25) that exist in JULIET. This results in five CWEs (the first five in Table V) that are mostly related to memory defects. We further select seven CWEs in JULIET, including well-known flaws like format string attack (ESTR) and race condition attack (TOC-TOU). All 12 CWEs are listed in Table IV, where “Abbr.” stands for the abbreviation of the corresponding CWE description. The detailed description can be found in Table V. Our experiments are launched on a 64-bit x86 Ubuntu 18.04 machine. For these 12 CWEs, JULIET offers a total of 30950 C/C++ programs. The CWEs with variants specific to Windows are not included.

Sharing the same concerns as [107], we further included REAL, a benchmark containing real-world vulnerabilities. In

particular, to enable fair comparison across tools, we reused a well-maintained CVE list [103] and rigorously preprocessed it. Specifically, we derived REAL from this CVE list by normalizing factors like build environments that could unfairly advantage certain tools (see details in Section V-B). The statistics are reported in Table II. Though REAL contains less software than JULIET, each real-world software is substantially more complex than those synthetic programs in JULIET. To clarify, the LOC count (341,559.9 per software) for REAL does not include libraries. Also, when using Q-SAST and NQ-SAST tools, we do not customize them (none of them are designed for extensive customization), and we use them in their standard mode. We report evaluation results at this step in Section V.

It is worth noting that both datasets can be built without any error, allowing the SAST tool to return analysis results regardless of whether it supports fuzzy parsing or not. As we introduced in Section II-B, fuzzy parsing enables analysis without building the entire software beforehand, and tools that lack support for fuzzy parsing would terminate and refuse to return any analysis results. Therefore, to provide a comprehensive and fair comparison, we do not include any broken programs in the dataset. We explain the studied 12 CWE cases in Table V for the reference.

C. Step ②: Analysis of Defect-Fixing PRs

We collect PRs from the GitHub repos of our studied Q-SAST tools to inspect each fixed defect. We categorize each PR depending on whether their fixed defects are FPs, FNs, or AFs. Though seven SAST tools are analyzed, only three tools, as listed in Table III, have PRs from the community. We only pick closed and merged PRs. Defects in these closed PRs have been accepted by developers, and such PRs usually contain rich information about specific defects, including code changes, links to related issues, and discussions. Based on the theme of the PRs, we divide them into different categories and only focus on two: bug fixing for AFs and inaccuracy enhancement for FPs/FNs. The former type is concerned with reporting unexpected crashes or exceptions that occur during the application of SAST tools, while the latter type focuses on reporting potential FPs or FNs in the analysis results provided by these tools. To facilitate comprehension of each PR, we search for PRs with adequate details to manually determine the root causes of fixed defects. Specifically, we pick a PR if: 1) it has more than five lines of code changed, 2) it has a description or test case provided, 3) it has related issues mentioned, and 4) it contains bug-relevant keywords (i.e., fix, defect, error, bug, issue, mistake, incorrect, fault, flaw, and false-positive/negative) in the tags or titles. For example, we encounter a false positive (FP) in CodeQL reported at [7], where the owner of the OpenConnect project [20] noted the erroneous labeling of code containing the “ctime_r” type. The developer of CodeQL acknowledged it as an FP, subsequently updating the queries to resolve it [6].

We gathered 1,288 PRs and identified 261 distinct bugs in three tools (CodeQL, Semgrep, Joern). Table III summarizes the studied PRs. With manual efforts, we classify each PR into addressing AF, FP, or FN. Following the same process in [120], two authors manually labeled each PR and cross-checked their

TABLE IV
ASSESSING Q-SAST/NQ-SAST VIA JULIET

CWE ID	Abbr.	#Instance	CodeQL			Joern			Semgrep			SonarQube			Mend			Coverity			Clang		
			TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN	TP	FP	FN
CWE78	OSCI	1720	308	0	1412	196	1332	1524	988	1374	732	8	231	1712	0	0	1720	200	114	1520	24	107	1696
CWE121	SBOF	4944	612	0	4332	863	1429	4081	3122	9153	1822	1415	1701	3529	0	0	4944	1707	530	3237	365	954	4579
CWE134	ESTR	2574	1230	1138	1344	472	2705	2102	396	854	2178	16	639	2558	0	0	2574	138	254	2436	12	70	2562
CWE190	IOF	3960	3393	51	567	1590	482	2370	979	1795	2981	159	1790	3801	0	0	3960	2040	898	1920	0	0	3960
CWE191	IUF	2952	1866	0	1086	1130	141	1822	703	1045	2230	114	1373	2676	0	0	2952	1512	611	1440	0	0	2952
CWE253	FRV	540	0	0	540	342	33	198	306	0	234	54	212	496	0	0	540	0	70	540	0	0	540
CWE367	TOCTOU	36	36	0	0	36	0	0	36	0	0	0	14	36	0	0	36	36	2	0	0	0	36
CWE416	UAF	460	306	0	154	42	18	416	10	0	450	360	913	117	0	0	460	336	570	124	216	262	174
CWE457	UVAR	950	452	1386	498	206	40	744	0	0	950	449	2028	501	0	0	950	386	1049	564	448	832	502
CWE476	NULLPD	330	218	0	112	185	0	145	58	70	272	198	278	143	0	0	330	192	90	138	193	108	118
CWE570	EAF	16	0	0	16	-	-	-	1	0	15	2	0	14	0	0	16	1	5	15	0	0	16
CWE676	UDF	18	18	0	0	18	0	0	0	0	18	0	7	18	0	0	18	0	3	18	0	0	18
Total		18500	8439	2575	10061	5080	6180	13402	6599	14291	11882	2775	9186	15601	0	0	18500	6548	4196	11952	1258	1963	16861

“Abbr.” stands for the abbreviation of CWE description. See Table V for detailed information of each CWE. For each cwe, we mark the highest TPs.

TABLE V
TWELVE CWES STUDIED IN THIS RESEARCH

CWE ID	Abbr	remark
CWE78	OSCI	Improper Neutralization of Special Elements used in an OS Command (“OS Command Injection”)
CWE190	IOF	Integer overflow or wraparound
CWE191	IUF	Integer underflow (wrap or wraparound)
CWE416	UAF	Use after free
CWE476	NULLPD	NULL pointer dereference
CWE121	SBOF	Stack-based buffer overflow
CWE134	ESTR	Use of externally-controlled format string
CWE253	FRV	Incorrect check of function return value
CWE367	TOCTOU	Time-of-check time-of-use (TOCTOU) race condition
CWE457	UVAR	Use of uninitialized variable
CWE570	EAF	Expression is always false
CWE676	UDF	Use of potentially dangerous function

“Abbr.” stands for the abbreviation of CWE description.

results to reach a consensus. Both authors are experienced in software security and SAST tools. We thus ensure the accuracy and the credibility of our findings. This step takes about 200 man-hours. Details of AFs/FPs/FNs are given and analyzed in Section VI.

Discussion on Eliminating Manual Efforts: One may wonder the possibility of eliminating manual efforts, as the manual effort is substantial and time-consuming. To clarify, the feasibility of reducing manual effort by ruling out irrelevant PRs using keywords have been introduced above as a potential solution. However, it cannot entirely replace manual efforts, particularly in the cases of bug-fixing PRs that lack a standard template and corresponding test cases. In such cases, manual investigation is necessary to comprehend the bug and associated PR through related discussions and code changes. Depending solely on automated classification by running a test may lead to many cases not being analyzed. Therefore, we use manual investigation to ensure the comprehensiveness and accuracy of our classification approach. We leave investigating a fully automated approach as future work.

D. Step ③: Exposing Defects With SAST-MT

Findings in step ① and step ②, as will be highlighted in Sections V and VI, reveal the challenge and demand to test Q-SAST tools. We design a metamorphic testing (MT)-based framework [53], SAST-MT, to test Q-SAST tools. In short, MT mutates test inputs using metamorphic relations (MRs)

to assert if certain program *invariant* properties are always satisfied [118]. For instance, to test $\sin(x)$, we can define an MR such that $\sin(x) = \sin(\pi - x)$ should always hold, no matter how we mutate x . A bug in $\sin(x)$ is detected whenever the MR is violated.

Starting from a seed program p_0 with vulnerabilities that are detectable by an Q-SAST tool, SAST-MT iteratively and incrementally mutates p_0 with MRs (each MR is a semantics-preserving transformation) and asserts if the same detection results are retained. If not, discrepancy results can expose FPs/FNs of this Q-SAST (see technical details in Section VII). Moreover, with SAST-MT, it becomes much easier to pinpoint the root causes of Q-SAST’s FPs/FNs: developers can simply diff two adjacently-mutated versions, where the latter one triggers FPs/FNs/AFs but the former one is correctly analyzed. With such an error-triggering input and flagged root causes on hand, developers can better debug their tool. As a proof-of-concept, we implement SAST-MT for CodeQL. SAST-MT should be applicable to other tools; see Section VII.

V. BENCHMARKING WITH CVE/CWE CASES

A. JULIET Cases

Evaluation results are in Table IV. Each tool(except Mend) detects some CWE instances in JULIET, as shown in the TP columns. CodeQL has the highest TPs for seven out of twelve CWEs and the “Total”. Nevertheless, it misses all CWE instances for CWE253 and CWE570 (see discussions below). Semgrep excelled for CWE78 (OS command injection) and CWE121 (stack-based overflow). We find that Semgrep’s queries correctly cover most instances of these two CWEs. However, Semgrep is overall aggressive, resulting in more FPs per CWE. CodeQL has no FPs for these two CWEs.

Joern outperforms CodeQL on CWE253, and has an identical number of TPs on CWE367. We find that both CodeQL and Joern have implemented a rich set of data flow tracking passes, particularly on taint-style dependency analysis. We believe Joern is less mature than CodeQL given the lack of community support. Joern’s query language design also seems a bit clumsy: we failed to convert CodeQL’s CWE570 query into Joern’s

query, and it is hard to write queries for CWE570 from scratch with APIs that Joern offers.

Clang, a leading NQ-SAST, is less comprehensive than those Q-SAST. It detects some instances of six CWEs, whereas Q-SAST tools detect many instances of 11–12 CWEs. According to our preliminary study (noted in Section III-B), Clang can outperform well-known SAST tools like Facebook Infer and CPPCheck. Furthermore, Coverity manifests a reasonably high accuracy in detecting CWEs, and it even surpasses Joern and Semgrep in some CWE IDs (e.g., CWE 476). We attribute this to the fact that Coverity is a mature NQ-SAST tool that has been meticulously maintained by community volunteers and the open source project team since 2006 [126]. Additionally, as a state of the art NQ-SAST [56], [79], Coverity is based on a large number of high quality research papers [46], [54]. However, Coverity cannot outperform the best of the three Q-SAST tools, and it is the only tool that requires users to wait 1–2 business days to receive the analysis report. We have no information about what analysis is undertaken on its server’s end, and its overall usage flow is more complex than the others.

SonarQube yields rich and helpful outputs (e.g., for enhancing code quality), but not all of its outputs are related to security. While it is valuable, its vulnerability detection may not be as thorough as other tools. Mend maintains 3,268 CVE patterns (as in Table I). We suspect that Mend will use all available CVE patterns to analyze a program. Nevertheless, Mend finds nothing over JULIET test cases. Therefore, it has no TP/FP. We hypothesize that it uses simple string-level signatures to match a CVE, e.g., matching the specific variable name. Our empirical exploration justifies this hypothesis; see details in Section V-B.

When cross comparing CWE detection results, both CodeQL and Joern show promising support for CWE134, CWE190, and CWE191. Analyzing these CWEs requires tracking the usage of sensitive data. CodeQL and Joern feature taint tracking to detect these CWEs. Contrarily, Semgrep, by matching the ASTs of vulnerable code patterns, manifests less accuracy and more FPs. This illustrates the strength of using data flow tracking. CodeQL fails to detect any flaws for CWE253. We carefully checked this case and concluded that CodeQL’s CWE253 query is defective. In short, this query only checks for statements that use an `HRESULT` as a boolean variable before using it in an `if` predicate. Because `HRESULT` is mainly a Windows data type, this query does not work on other platforms, including JULIET’s 540 instances of CWE253. Same observation is on CWE570, where CodeQL provides overly detailed queries.

Moreover, we conduct a Wilcoxon test [136] to test whether Q-SAST tools outperform NQ-SAST tools. To be specific, we test whether the TP rates of three Q-SAST tools (CodeQL, Joern, and Semgrep) outperform those of three NQ-SAST tools (SonarQube, Coverity, and Clang). Mend is not included to make a fair comparison, as it finds nothing in JULIET. It turns out to be that the p-value for the Wilcoxon test is 0.036, indicating that Q-SAST tools are better than NQ-SAST tools in terms of detecting the vulnerabilities. In sum, we present the following lesson before the next study:

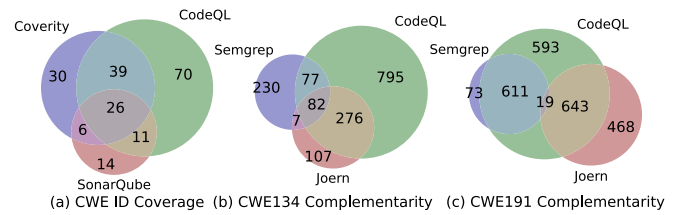


Fig. 5. Complementarity of various tools in detecting vulnerabilities at different levels.

Q-SAST can find many C/C++ flaws, outperforming industry-leading NQ-SAST. Study using real-world CVEs (Section V-B) reveals consistent findings, though also exposing under-documented characters and limits of Q-SAST.

Complementarity in Vulnerability Detection for Various Tools: When comparing different SAST tools, an intuitive enhancement is to combine them and explore the possible “synergistic effect” — suitable tools are employed when detecting different CWEs. We now analyze this from levels: *CWE ID level* and *CWE instance level* as follows.

The “CWE ID level” refers to the extent to which different tools support different CWE IDs. As shown in Table I, the CWE coverage of different tools varies substantially. We further compare three tools (CodeQL, SonarQube, and Coverity) based on their official websites, which explicitly provide each supported CWE ID. The result is shown in Fig. 5(a), where we find that although the number of supported CWEs varies across tools, they tend to support some common CWEs since they share more than half of CWE IDs with at least one other tool. Holistically speaking, it may be accurate to interpret that different tools are prioritizing similar kinds of vulnerabilities, potentially due to the fact that those cases are more common in practice. On the other hand, the overlapping indicates the potential that a (emerging) Q-SAST tools tool may promptly leverage prior experience from existing ones to calibrate its detectability.

The “CWE instance level” focuses on different tools’ detection results toward CWE instances belonging to the same CWE ID. We compare three Q-SAST tools (CodeQL, Joern, and Semgrep) on CWE134 and CWE191 in Fig. 5(b) and (c), respectively. For each CWE instance, we check whether it is detected by these three tools. The results show that the detectability of different tools on the same CWE instance varies significantly, providing the possibility of exploiting their complementarity to achieve better results. Due to space limitations, we only show the results of two CWEs here, and the remaining figures can be found on our website [27]. We note that similar observations can be made on those figures provided on the website.

1) Characteristics of FPs/FNs: After assessing the detectability across tools, identifying the root causes of FPs and FNs is critical. However, some Q-SAST tools are not open-sourced, and all give limited information to help users comprehend the causality of the findings. This impedes root cause analysis of each FP/FN, let alone thousands of FPs/FNs. The


```

rules:
- id: double-free
  patterns:
  - pattern: |
    free($VAR);
    ...
    SFREE($VAR);
  metadata:
  - CWE-415: Double Free"
  languages:
  - c
  severity: ERROR
(a) Semgrep query example

predicate outOfBoundsExpr(Expr expr, string kind) {
  if convertedExprMightOverflowPositively(expr)
  then kind = "overflow"
} // From CWE-190/IntegerOverflowTainted.qll
predicate convertedExprMightOverflowPositively(Expr expr) {
  exprMightOverflowPositively(expr) or
  convertedExprMightOverflowPositively(expr.getConversion())
} // From SimpleRangeAnalysis.qll
predicate exprMightOverflowPositively(Expr expr) {
  getUpperBoundsImpl(expr) > exprMaxVal(expr) or
  getUpperBoundsImpl(expr.(PostfixIncrExpr)) = exprMaxVal(expr)
or exprThatCanOverflow(expr) and not analyzableExpr(expr)
} // From SimpleRangeAnalysis.qll
(b) CodeQL query example

```

Fig. 6. Simplified query examples from Semgrep and CodeQL.

root cause groups originate from the JULIET benchmark, and are further processed and refined through our human evaluation. To address this, we turn to the JULIET benchmark, from which we identify and refine root cause groups with manual analysis. Specifically, each JULIET CWE has seed programs, and each seed program has multiple variants with mutations targeting specific code patterns compared to the original seed program. Therefore, when the tool produces inconsistent detection results on a seed program and its variants, we can determine that issues related to the variant type likely contribute to the root cause. We consolidate root-cause groups since each seed has at most 84 variants: to do so, two authors independently analyze the root causes for the inconsistent detection results, and then reconcile their findings. At this step, we mainly study FPs/FNs of the three best Q-SAST: CodeQL, Joern, and Semgrep.

For Semgrep, we do not inspect all its detection results, primarily due to limitations in Semgrep’s query abstraction. Specifically, Semgrep features regex-like queries, which are easy for developers to write but often yield inaccurate or incomplete analysis results. Therefore, once Semgrep fails to detect one CWE instance, it frequently fails in similar cases. Moreover, Semgrep produces many FPs, making investigation of every case infeasible. We randomly inspected 200 FPs of Semgrep. We point out that Semgrep’s FPs stem entirely from *inaccurate queries*. Consider a simplified query shown in Fig. 6(a), where Semgrep searches for a variable \$VAR that has been freed twice. However, it fails to recognize more complex but indeed safe situations, such as \$VAR being re-initialized (using new) via an alias of \$VAR. Though Semgrep provides some utilities to exclude FPs, we find that it is immature.

Further to Semgrep, we examine all FPs/FNs of CodeQL and Joern. For FPs, we also inspect the localized “vulnerable” program points. During human inspection, two analysts cross check their findings to reach a consensus. Examining all instances takes about 340 man-hours. This illustrates the difficulty of testing and debugging SAST. See Section VII on our effort to launch MT to test CodeQL. We categorize the root causes of FPs/FNs as follows. Notably, Q-SAST tools inherently utilize static analysis techniques, which inevitably results in some shared root causes of FPs/FNs with classic SAST tools, as will be reported below in this section. However, we also find that Q-SAST tools face unique challenges, such as the issue of “Incomprehensive/Aggressive Queries” (see FP₁/FN₁

below), due to its query-based design. We now elaborate on these categories.

Incomprehensive/Aggressive Queries – FN₁/FP₁: Queries, as manually-written vulnerability patterns, may often miss certain CWE instances due to the flexibility of C/C++ syntax. For both CodeQL and Joern, this is the most prevalent root cause of FNs. Consider the simplified example below,

```

fscanf(stdin, "%c", &data);      fscanf(stdin, "%c", &data);
//TP: CWE190 int. overflow      //FN: CWE190 int. overflow
char result = data+1;           char result = data*data;

```

where CodeQL missed detecting a CWE190 instance (integer overflow) in the right code snippet as its query (Fig. 6(b)) neglects the possibility for multiplication output to overflow. To be specific, the predicate in “IntegerOverflowTainted.qll” detects the integer overflow based on the built-in predicate named “convertedExprMightOverflowPositively” provided in “SimpleRangeAnalysis.qll”. Then, it recursively calls “exprMightOverflowPositively” and eventually relies on “getUpperBoundsImpl”, which fails to detect the multiplication that possibly causes overflow. Note that we do not show the details of “getUpperBoundsImpl” here, as it contains tens of overflow patterns across nearly one thousand lines of code. Interested readers can refer to our website [27] for details. Overall, as JULIET contains varied instances of each CWE, queries are often incomprehensive, making tools miss some corner case data types or arithmetic operations. Similarly, some “aggressive queries” treat secure C/C++ code snippets as vulnerable, even when they are safeguarded by user-defined checks (if statements). This is the primary reason for FPs.

Pointer Operations – FN₂: Pointer operations include pointer reference, dereference, and arithmetic expressions. Point-to analysis is a long-lasting challenge for static analysis. Complex usage of pointers can impede CodeQL and Joern from correctly tracking data flow facts, missing vulnerabilities.

Complex Data Structures – FN₃: CWEs may involve complex data structures including built-in types (e.g., union) and std structures like std::vector. CodeQL and Joern may encounter *under-tainting*, failing to thoroughly propagate taint information via complex data structures and incur FNs.

Inter-Procedure Analysis – FN₄ and FP₂: Inter-procedure analysis is inherently challenging. As reported in [8], CodeQL could fail to trace taint propagations across C/C++ files. Similarly, Joern would terminate taint analysis when the function call chain exceeds three functions in length. We however believe the FN and FP cases can hardly be addressed by directly tweaking these inter-procedure analysis “configurations”; for instance, if increasing Joern to perform taint analysis for longer call chains, it may incur more FPs or be trapped in performance issues. We leave further discussions on relevant issues (e.g., inter-procedure analysis) in Section V-B2.

Virtual Function Call – FN₅: C++ extensively uses virtual functions to resolve dynamic/late binding function calls. It is known that analyzing virtual function calls, which involve complicated point-to relations, is difficult. We observe that CodeQL and Joern offer limited, if any, support for point-to analysis in this scenario. CodeQL and Joern seem to ignore data passed into

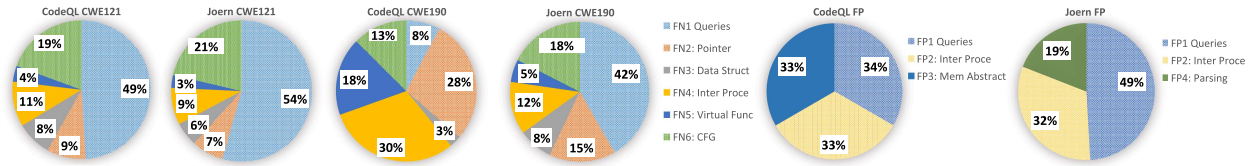


Fig. 7. Distribution of FP/FN root causes.

a virtual method and show no results for cases involving virtual function calls.

Intra-Procedural Analysis – FN₆: FN₆ occurs for complex intra-procedural analysis. While each JULIET program has a small number of functions, each function may have a large CFG, complex arithmetic expressions, and type castings. For instance, we find when `if` predicates have complex expressions or function calls, CodeQL and Joern may neglect some paths on the CFG and induce FNs.

Imprecise Memory Abstraction – FP₃: This is unique for CodeQL. We find that it may incorrectly treat global or static variables as uninitialized. This may lead to FP, treating legit usage of global/static variables as vulnerable (i.e., using uninitialized variables).

Incorrect Parsing – FP₄: Joern may fail to parse edge-case expressions, resulting in FPs. For instance, when a variable is compared with a macro (e.g., `i < INT_MAX`), Joern will treat the entire comparison as a variable instead of an overflow check. Then, the consequent usage of `i` leads to an “overflow” alert (as the overflow check is omitted), resulting into a FP.

Root Cause Distribution: Fig. 7 shows FP/FN root cause distribution. Due to limited space, for FN, we only plot CWE121/CWE190; others can be found on our artifact website. For CWE121/CWE190, FNs are attributed to all six discussed root causes. Both tools provide similar root cause distribution for CWE121. For CWE190, CodeQL has a notable number of FNs due to inaccurate point-to analysis. Overall, incomprehensive queries (FN₁) primarily contribute to Joern and CodeQL’s FNs for all CWEs. JULIET contains up to thousands of vulnerable programs with distinct syntactic forms for each CWE. Thus, Q-SAST tools’ queries often fail to cover all cases. Tuning queries to be general (reducing FNs without sacrificing FPs) is hard, often involving “trial-and-error.” See Section V-B1 for relevant discussion.

For FPs, we plot the distribution across all CWEs in Fig. 7. CodeQL has FPs for only three CWEs, CWE134, CWE190, and CWE457. We find that each FP’s root cause discussed above happens to explain full FPs in one CWE. Therefore, three FP root causes have an equal portion. FP root causes, except FP₃ which is unique to CodeQL, all exist in Joern’s FP cases across all CWE instances.

B. REAL Cases

Overview: Besides using the synthetic JULIET, we consider a well-maintained CVE list [103] to delineate how Q-SAST aids in detecting *real-world* defects. Each CVE should have a corresponding CWE ID. Therefore, we use the relevant CWE

queries. Also, as the community may already provide CVE-specific queries online, we search for corresponding CVE queries at our best effort. This illustrates how users may unleash community power in vulnerability search.

To enable fair comparison across tools, we rigorously preprocess REAL as follows: First, we subset the data to only contain CVEs associated with a specific CWE ID. This ensures that each vulnerability is mapped to a clearly defined weakness to detect. Second, we filter to only retain CVEs for which the software could be correctly built and the provided test inputs reliably triggered the vulnerability, allowing tools without fuzzy parsing capabilities to still analyze the software normally. This preprocessing enables us to thoroughly assess and compare the tools’ detection effectiveness on real software vulnerabilities under consistent and clear conditions. Originally containing 225 CVEs [103], REAL is reduced to 182 CVEs associating with a CWE ID after the first processing step, then further reduced to 136 CVEs meeting the second criterion. These 136 CVEs (corresponding to 14 unique CWEs) are our focus for detection.

For Q-SAST, CodeQL can correctly detect 13 CVEs with in total of 46 default queries. Joern can only discover CVE-2016-10261, an integer overflow in jasper 1.0.25, and Semgrep reports three CVEs related to the misuse of `memcpy`.

We also test NQ-SAST. Clang detects zero CVE in REAL, and Mend detects seven real-world CVEs. However, we hypothesize that Mend uses specific string-level signatures to pattern match a CVE. To justify our hypothesis, consider the following code snippet, which shows CVE-2019-13135 that can be detected by Mend.

```
1 // WhiteSource can detect this CVE in ImageMagick
2 BImgBuff = AcquireQuantumMemory(
3     ldblck, sizeof(*BImgBuff)); // CVE-2019-13135
4 if(BImgBuff==NULL) goto NoMemory;
```

After renaming variable `BImgBuff` into `k`, Mend failed to detect this CVE. Besides this case, we also explored the rest six (by tweaking variable and function names), and observed *consistent* findings. Thus, it should be accurate to believe that Mend uses string-level signatures to pattern match a CVE, making its detection capability of diverse real-world unknown flaws questionable. We thus omit studying it. As for Coverity, it refuses to disclose detailed reports on specific versions of open-source projects to us, since we are not the legit “owner” of these open-source projects. This forces us to abandon our study of it on REAL.

Inter-Procedural Analysis As we introduced in Section V-A1, a complete and accurate inter-procedural analysis is fundamentally challenging for SAST tools. This is primarily due to the following reasons: ① It is impossible to compute data flow

through standard library functions where the source code is unavailable. ② Certain behaviors are only determined dynamically at runtime, requiring extra effort to identify all potential call targets. ③ The intensive use of pointers in C/C++ programs can lead to the problem of variable aliasing, which in turn makes accurate analysis of interprocedural calls extremely complicated. ④ The data flow graph in inter-procedural analysis can become excessively large and computationally intensive to construct.

Consequently, the capabilities of Q-SAST tools in supporting inter-procedural analysis vary significantly. For example, Semgrep can only provide simple and elementary intra-procedural analysis as a trade-off between efficiency and accuracy. In contrast, both CodeQL and Joern provide a global (inter-procedural) data flow as well as a mature inter-procedural taint tracking analysis. However, as illustrated in Section V-B1, even CodeQL could fail to thoroughly analyze the inter-procedural calls in intricate situations such as when encountering pointer dereferences. In practice, the flow from source to sink would possibly be broken, and the developers of the Q-SAST tools encourage people to report this kind of bug. Before inter-procedural analysis capabilities are fully developed, temporary solutions with compensation functions [4] that introduce extra taint flows can enable the tools to bridge the gaps across functions and accomplish adequate inter-procedural analysis.

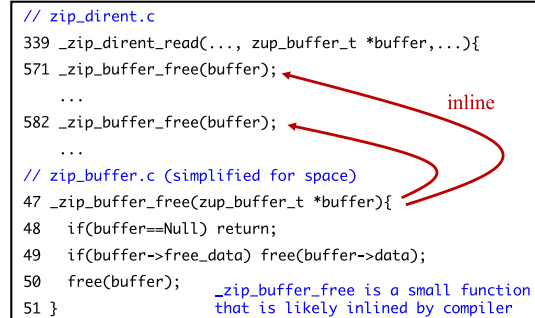
1) *Manual Enhancement*: Detecting 9.5% CVEs ($\frac{13}{136}$) is not satisfactory, though detecting real-world CWEs is inherently challenging for static analysis. Given that said, it may be too early to conclude that CodeQL has limited CVE detection support. Our observation (and experience in Section V-A) shows that *many FPs/FNs can be reduced when leveraging domain-specific knowledge to slightly tweak queries/software*: To our knowledge, many publicly available queries are written in a general form, which can facilitate further query tailoring for specific scenarios [26], [30], [112]. On the other hand, to our best knowledge, some “CVE-specific” queries are not completely released, because finding unknown vulnerabilities usually has high monetary rewards [87]. As third-party analysts, it seems inherently hard for us to objectively assess efforts to tune queries/programs. Nevertheless, we still offer three ways to address some of these deficiencies.

Reduce FPs by Tailoring Queries: CodeQL has many FPs when analyzing real-world software. We explore reducing FPs when analyzing CVE-2017-7598 in libTIFF 4.0.7. We first use a CVE-specific query found online [5] to search for this divide-by-zero vulnerability. This query is lenient, and when using it, we had 400 findings (one TP and 399 FPs). To reduce FPs, we make an assumption that real-world, well-developed software should have used `if` statements to guard obviously sensitive code. That is, developers may have sanitized this sensitive division statement with `if`, though the sloppy `if` statement neglects subtle cases, inducing the CVE. We add three LOC in the query to search division statements already sanitized by an `if` statement, and we successfully reduce the findings to 13 (one TP and 12 FPs). More cases about reducing FPs can be found in [21].

Reduce FNs by Tailoring Queries: CVE-2016-10251 (belonging to CWE190) in jasper 1.900.17 is an overflow induced

by integer left shift. CodeQL’s CWE190 query misses it. We find that CodeQL’s official CWE190 query captures overflow in various subtle arithmetic and logic expressions. However, it ignores CVE-2016-10251, which roots from integer left shift. We extend the standard query by mimicking how other arithmetic expressions are checked. We add six lines of code in the query and successfully detect this CVE. By tweaking queries in a similar manner, we successfully found twelve more CVEs (in total 13 more CVEs); see [21].

Reduce FNs by Tailoring Software: Besides tailoring queries, we have found it useful to tweak target software. As we mentioned in Section V-A1, the inter-procedural analysis capabilities of Q-SAST tools such as CodeQL are limited in precision due to various constraints faced by the original developers of the tools, including limitations in the design of the analysis queries and fundamental challenges in static analysis engines [8]. While CodeQL possesses some inter-procedural analysis abilities, it may fail to track data flows across function boundaries in certain cases falling outside its modeled scenarios. Therefore, we could make Q-SAST tools easier to match vulnerability patterns (codified in queries) with certain fragments in the target software by transforming software into more regulated forms. As an example, we tweak a function call in libzip (version 1.2.0) by manually expanding the `_zip_buffer_free` function inline at the call site, as follows.



```
// zip_dirent.c
339 _zip_dirent_read(..., zip_buffer_t *buffer,...){
571 _zip_buffer_free(buffer);
...
582 _zip_buffer_free(buffer);
...
// zip_buffer.c (simplified for space)
47 _zip_buffer_free(zip_buffer_t *buffer){
48 if(buffer==Null) return;
49 if(buffer->free_data) free(buffer->data);
50 free(buffer);
51 }
```

_zip_buffer_free is a small function that is likely inlined by compiler

By inlining the `_zip_buffer_free` function, the taint analysis becomes capable of tracking taint flow and identifying CVE-2017-12858. Additionally, we also find that tailoring code for function inlining is scalable, as we successfully found three more CVEs (in total 4 more CVEs); see [21].

2) *Discussion*: Though CodeQL can detect some real-world flaws, it may be less desirable to expect an *out-of-the-box* usage of Q-SAST in real-life scenarios. The CWE/CVE queries online are often intentionally simple or conservative, requiring program-specific information to reduce FPs/FNs [25], [26], [30], [112].

To improve Q-SAST, it is rational to expect a “human-in-the-loop” process, which eases the burden for users to tailor queries/software when needed. It is however undesirable, from the users’ perspective, to expect a black-box Q-SAST. For example, while CodeQL can detect 13 CVEs from REAL using its shipped queries and queries online, we show that another 17 CVEs (in total 30 CVEs, count $22.1\% = \frac{30}{136}$ CVEs in REAL) can be found by reasonably tweaking queries/software. Thus, we believe designing an *interactive* analysis pipeline to enhance

TABLE VI
CATEGORIZING DEFECT-FIXING PRs

	FP ₁	FP ₂	FP ₃	FP ₄	FN ₁	FN ₂	FN ₃	FPN ₁	FPN ₂	AF ₁	AF ₂	AF ₃	Total
CodeQL	4	9	30	0	0	0	0	0	1	0	0	0	44
Semgrep	4	4	0	11	34	3	27	9	6	28	1	9	136
Joern	0	0	0	0	5	1	0	5	2	6	1	0	20

Q-SAST appears to be a promising next step. In sum, we make the following advocacy:

Q-SAST is mostly anticipated to aid rather than replace users. As also mentioned in [131], they do not try to offer an end-to-end, black-box solution. Considering such interactive procedures, Q-SAST should offer more information to aid users, e.g., showing “confidence scores” for their findings, or flagging code fragments where static analysis is difficult or inaccurate. Such information should help users tune their queries or software to reduce FPs/FNs when analyzing large real-life software. To date, Q-SAST tools haven’t revealed how they make decisions yet.

Furthermore, though offering a black-box detector is not the main goal, we believe that there is still a considerable space to improve the status quo of Q-SAST. From the **Reduce FNs by Tailoring Software** paragraph above, we envision that compiler optimizations like inlining can likely enhance de facto Q-SAST. Q-SAST tools appear not to extensively leverage compiler-level optimization techniques, and they feature limited inter-procedural data-flow analysis capability (likely reducing analysis overhead). We discuss this point further and summarize lessons in Section VI-A.

Threat to Validity: Enhancement from Developers: Tweaking queries/programs to reduce FPs/FNs is not as simple as it appears. Real-life vulnerabilities are overall complex to detect. Authors are skillful in using Q-SAST. However, we lack domain knowledge of the analyzed software (e.g., complex media software). Even worse, the causality of CVEs are often obscure to us. Again, as clarified in **Reduce FPs by Tailoring Queries**, users of Q-SAST, who may be developers or security analysts of the target software, are presumably more knowledgeable on which code fragments are buggy; they can perform analysis accordingly [26], [30], [112].

VI. ANALYSIS OF DEFECT-FIXING PRs

Table III has summarized PRs of three representative tools, categorizing defect-fixing PRs into FAs, FPs, and FNs. We inspect each PR-fixed defect and put them into 12 categories, as in Table VI. A large number of defect-fixing PRs for these tools indicates that the community actively contributes to their quality. Most CodeQL’s PRs are about FPs, not FNs/AFs. CodeQL is widely-used in open-source projects, and we suspect its users (developers of open-source projects) are sensitive about FPs that may undermine their projects’ reputation. Defects are discussed in seven groups as follows.

① *Buggy Software Parsing – FP₁/FN₁/FPN₁:* Overall, the tools we studied seek to create (fuzzy) parsers for various

languages. This seems ambitious, given the complexity of many statically compiled languages like C/C++. Many PRs are issued to repair FPs and FNs caused by parsing errors. Considering the following example:

```

1 // success case                1 // failed case
2 int a[]={0, 1, 2}              2 int a[] {0,1,2};
3 for (int i : a) { ... }        3 for (int i : a) { ... }

```

where both code snippets are grammatically correct. However, the array initialization (line 2) in the right code snippet is less common. Joern incorrectly parsed this initialization, and consequently warned line 3 as “using uninitialized variable a”, inducing a FP.

Parsers may also ignore code snippets written in corner-case syntax, leaving dummy basic nodes in parsed syntax trees with no data-flow or control-flow information attached. Tools with such flaws may overlook vulnerabilities (i.e., FNs). There are additional 14 (9+5) PRs fixing parser defects that can simultaneously cause FPs and FNs (FPN₁ in Table VI).

② *Inaccurate Static Analysis – FP₂/FN₂/FPN₂:* Static control- and data-flow analyses are core components of all SAST tools. We find that modern Q-SAST tools have good engineering quality. Nevertheless, static analysis passes are overall challenging and error-prone to implement. In particular, as Q-SAST tools may launch taint analysis, we find over ten PRs fixing *over tainting*, a classic hurdle inaccurately propagating inputs and inducing FPs. We also find 4 (3+1) PRs fixing inaccurate analyses inducing FNs, and 9 (1+6+2) PRs fixing analysts simultaneously inducing FPs and FNs.

③ *Aggressive Query – FP₃:* A new query may require gradual tuning to be complete and accurate. As expected, PRs are created to amend queries that are too aggressive. Consider a simplified example below:

```

1 bug_len = (int)strlen(longer_buffer);
2 short_buffer = malloc((size_t)buf_len);

```

where CodeQL warns for “no space to null-terminate the string”. However, since `longer_buffer` always exceeds `short_buffer`, this code is safe because there is always room for a null-terminator. CodeQL developers fixed the buggy query by considering the dataflow facts. We found 30 PRs fixing aggressive queries that cause FPs.

④ *Buggy Query Optimization and Search – FP₄/FN₃:* FP₄ and FN₃ root from query compilation/optimization and using queries to search codebases. Semgrep queries can be concatenated with logic operators to search several patterns simultaneously. Semgrep heuristically reduces the concatenated queries. Nevertheless, such heuristics may be unreliable and lead to FPs/FNs. Also, while CodeQL and Joern use SQL-like queries to search standard database tables, Semgrep converts queries into

TABLE VII
ATTRIBUTING DEFECT-FIXING PRs TOWARD DIFFERENT COMPONENTS OF Q-SAST TOOLS

	Parser	Analysis				Query	Matching	Others	Total
		data-flow (e.g., taint analysis)	control-flow analysis	constant propagation	others				
CodeQL	4	8	2	0	0	30	0	2	46
Semgrep	79	3	1	5	0	0	55	46	189
Joern	15	4	1	0	0	0	0	6	26

ASTs and matches the converted AST with the target software (also in ASTs). This step seems error-prone, leading to defects fixed by PRs.

⑤ *Crash/Exception During Parsing* – AF_1 : We group crashes and exceptions thrown during parsing in AF_1 . We find that Q-SAST tools' parsers are under pressure to cover full sets of syntax in different languages. This is ambitious, and several PRs are filed to fix parsing crashes/exceptions.

⑥ *Performance Issue During Analysis* – AF_2 : Scalability is one key challenge for static analysis. According to our observation, the analysis speed of popular Q-SAST tools may be slow for large software; see **Leveraging Database Optimizations** in Section VIII for discussion. Joern and Semgrep have two PRs to optimize taint analysis when analyzing large codebase.

⑦ *Crash/Exception in Search* – AF_3 : Crashes/exceptions during codebase search are grouped in AF_3 . This is primarily on Semgrep.

A. Root Cause Analysis of PR-Fixed Defects

Besides collecting and analyzing PRs, we further analyze the root causes of each fixed defect. To do so, we manually review descriptions in each PR. Table VII attributes PR-fixed defects to the technical pipeline of Q-SAST tools. "Others" include loggers, setup scripts, and other less specific issues like non-UTF8 character decoding.

Parser seems most buggy in these tools. All these Q-SAST tools are ambitious to be multilingual and parse different languages into their customized representations. This is technically challenging and error-prone. Also, despite its high engineering quality, CodeQL contains 30 query-related PRs. The parser and query engine of CodeQL are not open-sourced, missing eyeballs to inspect. We conclude:

Parsers of modern Q-SAST are challenging to implement, given that they are designed to offer multilingual support. They are prone to faults that cause FPs, FNs, and AFs.

Compiler Frameworks: Compiler ecosystems like LLVM [86] should boost the development of Q-SAST. First, in response to the likely buggy parsers (discussed in Section VI-A), the LLVM ecosystem has provided mature front-ends for several languages, saving Q-SAST the trouble of building parsers from scratch. Second, while de facto Q-SAST tools seemingly lack code optimizations (e.g., function inline, as shown in Section V-B), using LLVM allows leveraging the whole set of compiler optimizations achieved over decades. Using some advanced point-to-analysis infrastructures also becomes possible [125]. Moreover, LLVM compiler front-ends convert programs into a unified

IR, which should ease analysis in multilingual scenarios. It is unclear if Q-SAST, which are often closed-source, have their own IRs.

Nevertheless, some practical hurdles exist. First, not all languages, particularly dynamic languages like Python, have their mature front-ends available in LLVM. Also, using compiler IRs necessitates a building environment (software must be compilable), which is contradictory to "fuzzy parsing" promoted by many Q-SAST tools. Some vulnerabilities (e.g., uninitialized variables) may also become too subtle to capture in IRs. Using compiler frameworks may cause legal concerns, especially when commercializing Q-SAST products.

Technically speaking, reusing compiler frameworks like LLVM may relieve many engineering hurdles of Q-SAST tools. Compiler frameworks may however raise certain concerns on the application scope and legislation.

B. Comparison With Findings in Section V

When benchmarking Q-SAST tools using JULIET, we also summarize the root causes of FPs/FNs. Findings here are mostly aligned with Section V. It may be thus proper to assume that study in Section V reflects real usages of Q-SAST. Particularly, comparing findings in Section V-A with FP/FN root causes of PR-fixed defects, FP_{2-3} and FN_{2-6} in Section V-A correspond to "② Inaccurate Static Analysis", whereas FP_4 corresponds to "① Incorrect Parsing Results". "④ Buggy Query Optimization and Search" has no corresponding findings in Section V-A, as we did not investigate Semgrep's FPs/FNs in Section V-A.

FP_1/FN_1 in Section V-A imply queries are either incomprehensive or aggressive. Interestingly, while FN_1 — incomprehensive queries — is the primary reason for Joern and CodeQL's FNs (see Fig. 7), we find **no** PRs fixing queries leading to FNs; ③ is exclusively for FPs, fixing too aggressive queries. When users use Q-SAST to scan their code, they often have no ground truth, meaning that it is hard to decide if a FN exists. However, users can look into all positive findings and file a PR if those findings are believed as FPs.

Comparing characters of PR-fixed defects and remaining pitfalls exposed in our CVE/CWE detection study, we notice that most defects have raised awareness of the community, and they are gradually fixed via PRs. Nevertheless, FNs, though widely exist, have *not* been primarily fixed. This may be due to the difficulty of providing ground truth, which advocates the need of creating better test suites.

VII. TESTING CODEQL WITH SAST-MT

Design Consideration: Mutating Software According to Findings in Sections V and VI: We present a proof-of-concept metamorphic testing (MT) [53] framework, SAST-MT, to detect FPs/FNs in Q-SAST and thus improve its vulnerability detection capability. We test the latest version of CodeQL given its popularity. Though CodeQL accepts queries and C/C++ programs as its inputs, we particularly mutate C/C++ programs (not queries). This way, we stress the program parsing and analysis phases of CodeQL, which are fundamentally challenging to implement and seem buggy, according to findings in Sections V and VI.

Design Goal: SAST-MT aims to benefit Q-SAST developers. SAST-MT uses MT to automatically uncover FPs/FNs of Q-SAST. Once preparing seed programs with known vulnerabilities, testing Q-SAST with SAST-MT does not need further manual efforts or annotated datasets. The reason is that SAST-MT *incrementally* and *iteratively* mutates a seed using pre-defined MRs (see below for the details). Let p_{i+1} be a program mutated from seed p_0 (vulnerabilities in p_0 are detectable by CodeQL) for $i + 1$ iterations, we rely on an invariant property, vulnerabilities in mutated programs are still detectable, to form the testing oracle to test CodeQL. In particular, a FP is found, when CodeQL detects more vulnerabilities on p_{i+1} than that of p_0 . Similarly, a FN is found when CodeQL detects fewer vulnerabilities on p_{i+1} than that of p_0 . Also, when CodeQL has FPs/FNs on p_{i+1} , but yields identical results on p_i and p_0 , developers can diff p_{i+1} and p_i to pinpoint code changes on p_{i+1} that presumably serve as the root cause of the detected FPs/FNs on p_{i+1} .

Clarification: We wish to clarify the design consideration of SAST-MT from two aspects. First, we do not claim credits from the “research novelty” over SAST-MT; metamorphic testing has been well known and studied for a long time by the community. Rather, we present SAST-MT as a very practical and handy tool, to facilitate the Q-SAST community and developers on automatically uncovering FPs/FNs of Q-SAST. As illustrated in our previous study, FPs/FNs are pervasive in the Q-SAST tools, and we see a demand from the community on automatically uncovering FPs/FNs. Second, SAST-MT does not directly address the issue of “improving the efficiency of vulnerability detection” of Q-SASTs. This is a very challenging task that requires fundamental improvement in static program analysis technique and we deem it as a long-term research goal. Rather, SAST-MT forms a very practical tool to help developers quickly and efficiently perform quality assurance on Q-SAST tools. This is generally the first step to systematically assess the efficiency of Q-SAST tools, and therefore, developers can rely on SAST-MT to depict a comprehensive and accurate picture of the status quo of Q-SAST tools, and accordingly improve their efficiency with more efforts and guidance.

Metamorphic Relations (MRs): SAST-MT designs fifteen MRs $m \in \mathcal{M}$, where each MR m specifies a *semantics-preserving* transformation over a seed program or its mutated outputs. Each m transforms programs at different hierarchies, including program types, structures, pointers, and other data flow facts. These methods are inspired by code diversification

TABLE VIII
FIFTEEN METAMORPHIC RELATIONS (MRs) STUDIED IN THIS RESEARCH

MR id	MR Abbr	Description
1	ETC	Functional equivalent type changes
2	Bool2num	Rewrite boolean logic to numbers
3	IfOR	Adding OR logic to if statement
4	IfAND	Adding AND logic to if statement
5	ReAPI	Result equivalent API rewrite
6	ReNum	Rewrite numeric returns
7	Remat	Rewrite ++ OR --
8	Ref1	Adding more reference to a variable TYPE 1
9	Ref2	Adding more reference to a variable TYPE 2
10	Loc2Glo	Change a local variable to global variable
11	GRV	Generate a totally random variable to global
12	RVF	Redefine a variable within function
13	Loop	Embed a statement with for single iteration for loop
14	MAXVar	Return numeric variable with max min
15	MAXNum	Return number with max min

“MR abbr.” stands for the abbreviation of MR names.

TABLE IX
FINDINGS OF LAUNCHING SAST-MT TOWARD CODEQL

#Test Inputs	#FP	#FN
100	17	228

schemes [85]. As mentioned above, they are designed to perform incremental mutation to ease debugging. The details of the fifteen MRs along with brief descriptions are provided in Table VIII. Interested readers can refer to the project website [27] for the concrete implementation and illustrative code examples before and after application of each MR. After mutating each (already mutated) seed p_0 with known vulnerabilities, we assert if any discrepancy is found from the results of CodeQL.

Setup: We used the Google Code Jam (GCJ) [14] dataset to prepare seeds. GCJ comprises source code from solutions to GCJ programming challenges. GCJ contains 740,263 C/C++ programs, and we use the default setting to compile each program. During compilation, we enable address sanitizer (ASan) [119]. We then use Google Honggfuzz [16] to fuzz each ASan-enabled GCJ C/C++ code. When fuzzing each program for 30 minutes, the *least* coverage per program has reached over 80%. We thus deem the coverage is satisfactory. This step helps to identify 10,307 programs containing at least one crash (e.g., buffer overflow). We then randomly select 100 programs to form the seeds where we require that each seed program satisfies: 1) CodeQL can detect its vulnerabilities (we compare CodeQL’s report with ASan’s reports to confirm that), and 2) the vulnerabilities are in the tested GCJ code rather than others.³ GCJ programs are simple, and we find that CodeQL can mostly detect the ASan-revealed flaws.

Results: The testing is launched on a machine with AMD Ryzen Threadripper 3970X CPU with 256 GB RAM. We mutate each seed for one hour (resulting in an average of 70.9 mutated programs per seed), and confirm the analysis consistency of CodeQL on each mutated program. As shown in Table IX, we find 17 FPs and 228 FNs, indicating the effectiveness of SAST-MT in stressing CodeQL. Considering the following (simplified) case:

³We encountered a recent flaw in kernel [32], not in GCJ code.


```
// one statement in  $p_i$ 
next[id] = &node[p]; // TP: buffer overflow

// the mutated statement in  $p_{i+1}$ 
next[std::min(id, 2147483647)] = &node[p]; // FN
```

where SAST-MT mutates program p_i by wrapping the usage of `id` in one statement of p_i with `std::min` and produces p_{i+1} . `std::min` is also fed with the maximal `int32`. Though CodeQL correctly detects a buffer overflow in the original statement in p_i (when `id` is greater than the size of `next`), CodeQL detects nothing in the mutated statement in p_{i+1} . We doubt that CodeQL cannot propagate taint through `std::min`. We report this finding to CodeQL and are waiting for their confirmation. See more findings of SAST-MT at [27].

To enhance the C/C++ detectability, we design SAST-MT, an MT framework that allows developers to automatically test Q-SAST for FPs/FNs at a reasonable cost.

VIII. DISCUSSION

One limiting factor for this study is that popular Q-SAST tools in use today are mostly closed-source, reducing the evaluation to black-box criteria, PR study, and functional testing. This tends to result in relatively straightforward examples throughout the paper. And we agree that it is generally hard to understand the causes for Q-SAST failures without a white-box perspective. Nevertheless, we believe that this research targets an important, under-explored, yet challenging field of benchmarking Q-SAST. The rest of this section first discusses inspiring and important observations made in this study. We then conclude this section with limitations of our empirical study methodology.

Detecting C/C++ Vulnerabilities with Q-SAST: Are We There Yet? Though software static analysis is inherently undecidable, it has been intensively studied in recent decades by industry and academia. In recent years, we've seen the industry and open-source community embrace and contribute actively to the development of Q-SAST. Despite these overall enthusiasms, the research community has not gained sufficient information to evaluate its quality and strength, nor has it been sufficiently disclosed to its users (most Q-SAST tools are not fully open sourced). This paper aims to understand the status quo of Q-SAST. We demystify current capabilities, as well as the additional effort required to improve them in daily C/C++ vulnerability detections. Their establishment may eventually result in breakthroughs that ease many security studies.

Our study shows that modern Q-SAST can find reasonably more vulnerabilities, outperforming industry-leading NQ-SAST regarding C/C++ vulnerability detection (Section V-A and V-B). These findings establish a solid foundation for the future application of Q-SAST in code auditing and security analysis. Given that said, we make several likely under-documented observations about Q-SAST that may not have received enough attention thus far. First, users should not anticipate finding full sets of CWE queries online at this moment, and it requires community-led initiatives to create a query (Section III-B). Second, while most issues exposed in our CVE/CWE evaluation

seem known to Q-SAST developers, FNs are harder to find and are not substantially fixed (Section VI-B). FNs are particularly undesirable in security analysis, since they indicate that certain attack vectors are overlooked. Accordingly, SAST-MT automatically detects FPs/FNs (Section VII), by stressing software static analysis passes that are error-prone (Section VI-A). We also advocate Q-SAST leveraging mature compiler frameworks, hence easing the process of implementing complex parsers or static analysis passes (Section VI-A). Also, in practice, one should not expect an “out-of-the-box” usage of Q-SAST. Q-SAST aid rather than replace users. To reduce FPs/FNs, users may need to tweak queries/programs with domain knowledge. With this regard, Q-SAST should provide more information to aid users (Section V-B2).

Future Work: Better Test Suites: We employ JULIET and REAL in our study. JULIET, as a synthetic dataset, provides many test cases. Its test cases are however less comprehensive than real-world software. For instance, JULIET has limited C++ test cases and no C++11 features. REAL, though including real-world software with known CVEs, has a limited type of flaws. Besides, we find that FPs/FNs found in both JULIET and REAL are hard to understand, let alone use for debugging. As noted in Section V, we spend considerable manual efforts to inspect the causes of FPs/FNs in Q-SAST outputs. Most prominent test suites for bug detectors are synthetic [51], [64], [94], [100], [139]. Recent efforts to produce more realistic defects have focused mostly on benchmarking fuzz testing and symbolic execution engines [77], [82], [110], [115].

We believe *injecting realistic vulnerabilities into synthetic programs of diverse forms* would be desirable to benchmark Q-SAST. While real-world software (with known CVEs) can better stress Q-SAST, large software makes it hard to understand FPs/FNs and troubleshoot. Looking ahead, it is vital to further improve JULIET, such as supporting new CWEs, enhancing the realism of injected flaws [77], [82], [110], [115], and adding new language features. Also, to ease localizing FP/FN root causes, SAST-MT can be employed to prepare *incrementally mutated* test cases. Though JULIET prepares “good” and “bad” variants (Section IV), their differences are still too large to easily localize root causes of FPs/FNs via diffing.

Future Work: Leveraging Database Optimizations: To our observation, de facto Q-SAST tools have not extensively explored optimization techniques provided by the database community. Q-SAST queries typically consist of a collection of predicates, which can be expensive to evaluate for large codebases. While analyzing synthetic test cases in JULIET is generally speedy, CodeQL spends several minutes processing *one* query toward databases derived from real-world software like libpcap and FFmpeg in REAL. The software parsing, analysis, and database creation time may range between 36 minutes (for FFmpeg) to 2.76 hours (for qemu). Database creation is usually a one-time effort. However, reducing query time is needed since users would use *multiple* new or enhanced queries to *repetitively* check databases [29]. Q-SAST can incorporate database query optimization techniques such as predicate pushdown [88] to increase query speed. Also, it is likely feasible to optimize query execution engines to prioritize suspicious code snippets

using proxy schemes [81] and offer best-so-far results within user-specified time budgets.

Limitation of Our Empirical Study Methodology: The ground truth is gathered in our study by manual effort (Section V), inputs from users/developers (Section VI), or via metamorphic testing (Section VII). In Sections V and VI, two authors cross check the manually-summarized findings. Still, a possible threat is that all three steps (including metamorphic testing) may produce erroneous “ground truth,” thereby exaggerating the capabilities of Q-SAST. Avoiding this issue is challenging; one possible mitigation is to cross-compare the outputs of Q-SAST tools and to get expert assistance in determining whether output inconsistencies are due to limitations or bugs. We leave exploring this direction for future work.

Another limitation is that our study only focuses on C/C++ language at present. We made this design decision due to the fact that C/C++ are deemed as unsafe and C/C++ software may contain a variety of severe vulnerabilities, primarily related to memory issues. Also, SAST tools for C/C++ software appear to be mature. It is possible that our empirical pipeline and the findings are unable to adapt to software written in other languages. However, we point out the chance is slim, given that our study pipeline is mostly platform- and language-independent. As a result, we anticipate that our study is applicable to other contexts. Section IX further reviewed relevant studies on SAST in the context of detecting specific privacy leakage vulnerabilities from Java and Android apps.

IX. RELATED WORK

We have reviewed de facto Q-SAST and NQ-SAST in Sections II-C and III-B. In what follows, we present further literature reviews, including related benchmarks and datasets for evaluation, various techniques for improving the detectability of SAST tools, and other related works.

Benchmarks and criteria: To date, various perspectives have been considered to benchmark SAST tools [47], [50], [73], [75], [97], [107], [108], [109], [122], including designing appropriate benchmarks, defining proper assessment criteria and using expert knowledge to benchmark SAST tools. For instance, Nunes et al. [107] propose a general approach to design benchmarks that contain real vulnerable web applications, which is representative of scenarios with different levels of criticality. In addition, Shiraishi et al. [122] propose test suites containing 9 defect types, as well as four evaluation metrics to benchmark SAST tools. Furthermore, Martinez et al. [97] propose a novel approach that considers multiple metrics, scenarios, and expert opinions to benchmark security tools. By aggregating expert judgments on the importance of metrics, this multi-metric and multi-expert approach can achieve a more comprehensive assessment of tool effectiveness. In this work, we use a synthetic dataset and a real-world one to thoroughly evaluate and compare Q-SAST tools under various scenarios.

SAST Tools: To date, a number of recent research works have surveyed SAST tools [37], [41], [52], [59], [70], [76], [89], [91], [93], [98], [99], [102], [114], [116], [117], [129], [132], [133], [134]. Existing works primarily focus on NQ-SAST, without presenting a systematic and specific benchmark toward Q-SAST.

Also, most of them access *bug detection* rather than vulnerability detection, or leverage only a limited number of CWEs [91]. In Section III-B, we extensively refer to those works when selecting proper SAST tools.

Vulnerability detectability for SAST: Recent works have empirically studied the vulnerability detectability of Q-SAST/NQ-SAST [39], [40], [48], [96]. Nevertheless, they primarily measure the detectability of a number of specific vulnerabilities, such as private data leakage [40], [48] or crypto-API misuse [39]. In comparison, we benchmark the detectability of a wide range of C/C++ vulnerabilities, such as the infamous buffer overflow, use after free, NULL pointer deference, and others. In this regard, our study differs from [39], [40], [48]. Mantovani et al. [96] propose a case study of SAST, but only consider 10 real-world cases, limiting systematic analysis. On the contrary, we study tens of thousands of synthetic cases and hundreds of CVEs, enabling more persuasive findings. Moreover, we concentrate mostly on C/C++, where memory-related vulnerabilities are pervasive due to the unsafe language design. In contrast, [39], [40], [48] mainly study Java and Android apps.

SAST Enhancement: Many works have proposed methods to enhance SAST tools by reducing FPs or FNs. Overall, these enhancement techniques can be roughly categorized into two types: (1) pattern-based [38], [80], [138] and (2) machine learning-based [55], [65]. For pattern-based methods, the primary goal is to use patterns to recognize potential FPs and remove them, thus improving SAST tools. For example, Aloraini et al. [38] show SAST tools can assess software quality without manual review by using “pattern-matching” techniques that significantly improve analysis. Similarly, Yang et al. propose Priv [138] to help developers better utilize SAST techniques. Moreover, patterns are not limited to code. For instance, Imtiaz et al. [80] study frequent alert patterns in open source software on Coverity scan to improve SAST usability. For machine learning-based methods, the main idea is to leverage the machine learning model’s ability to understand the code semantics and reduce FPs, thereby enhancing SAST performance. For example, Chow et al. [55] propose Fluffy, which is on top of CodeQL. Fluffy applies machine learning models to understand the natural language information involved in a taint flow and filter out the “unexpected” flow, enhancing the analysis quality. Moreover, USpec [65], a probabilistic model learned via unsupervised learning, is used to identify the potential API aliasing problem in SAST.

Learning-Based Vulnerability Detectors: There is a rich set of code embedding literatures that aim to extract software, either in source code or binary code format, into analysis-friendly numerical representations with hierarchical syntax or semantics information [35], [36], [45], [63], [90], [111]. Often, the end goal is to train machine learning or deep learning models to predict the existence of vulnerabilities hidden in the software. Nevertheless, as pointed out by a recent study [78], such learning-based vulnerability detectors often omit to flag even simple stupid bugs (SStuBs), indicating less capability in comparison with modern static analysis-based detectors like Q-SAST and NQ-SAST. We leave it as one future work to explore using methodology presented in this paper to benchmark modern deep learning-based vulnerability detectors in a comprehensive manner.

X. CONCLUSION

We conducted the first empirical study to demystify Q-SAST tools' vulnerability detectability. We evaluate their FPs/FNs using synthetic CWE and real-world CVE test suites. We also look into their bug-fixing PRs to understand root causes of fixed detects and comparing them to the issues found in our CWE/CVE study. We then design SAST-MT, an automated tool for testing CodeQL, exposes its FPs/FNs and simplifies debugging. We highlight lessons for users and developers to use and to enhance Q-SAST tools.

ACKNOWLEDGMENT

We thank anonymous reviewers and editors for their valuable feedback.

REFERENCES

- [1] "2022 vulnerability statistics report," 2022. [Online]. Available: <https://www.edgescan.com/2022-vulnerability-statistics-report-lp/>
- [2] "CodeQL bounty programs," 2021. [Online]. Available: <https://securitylab.github.com/bounties/>
- [3] "CodeQL C/C++ CWE coverage," 2021. [Online]. Available: <https://codeql.github.com/codeql-query-help/cpp-cwe/>
- [4] "CodeQL C/C++ dataflow," 2021. [Online]. Available: <https://codeql.github.com/docs/codeql-language-guides/analyzing-data-flow-in-cpp/>
- [5] "CodeQL CVE queries," 2021. [Online]. Available: https://github.com/elManto/SAST_on_Decompilers/tree/main/CVE
- [6] "CodeQL false positive fix record," 2023. [Online]. Available: <https://github.com/github/codeql/pull/6540>
- [7] "CodeQL false positive issue," 2023. [Online]. Available: <https://github.com/github/codeql/issues/6451>
- [8] "CodeQL interprocedural concerns," 2023. [Online]. Available: <https://github.com/github/codeql/issues/13896#issuecomment-1667487833>
- [9] "CodeQL introduction," 2022. [Online]. Available: <https://help.semmle.com/QL/ql-training/cpp/intro-ql-cpp.html>
- [10] "CodeQL security lab," 2022. [Online]. Available: <https://securitylab.github.com/research/>
- [11] "Common weakness enumeration," 2022. [Online]. Available: <https://cwe.mitre.org/index.html>
- [12] "Contributing to CodeQL," 2022. [Online]. Available: <https://github.com/github/codeql/blob/main/CONTRIBUTING.md>
- [13] "CVE database," 2023. [Online]. Available: <https://cve.mitre.org/>
- [14] "GCJ-dataset," 2022. [Online]. Available: <https://github.com/Jur1cek/gcj-dataset>
- [15] "GitHub security apps," 2022. [Online]. Available: shorturl.at/rCP67
- [16] "honggfuzz," 2022. [Online]. Available: <http://honggfuzz.com/>
- [17] "KLEE execution time," 2021. [Online]. Available: https://www.dropbox.com/s/s1atcgwvby5q2hb/klee_table.pdf
- [18] "Leveraging machine learning to find security vulnerabilities," 2021. [Online]. Available: shorturl.at/itzKZ
- [19] "Mend vulnerability database," 2021. [Online]. Available: <https://www.mend.io/vulnerability-database>
- [20] "OpenConnect introduction," 2021. [Online]. Available: <https://www.infradead.org/openconnect/>
- [21] "Real dataset information," 2021. [Online]. Available: <https://sites.google.com/view/qsast-survey/real-world-cves>
- [22] "Semgrep queries," 2021. [Online]. Available: <https://twitter.com/r2cdev/status/1511080792230752257>
- [23] "Shiftleft OverflowDB," 2021. [Online]. Available: <https://github.com/ShiftLeftSecurity/overflowdb>
- [24] "SonarSource queries," 2021. [Online]. Available: shorturl.at/bnrAE
- [25] "Variant analysis," 2021. [Online]. Available: <https://help.semmle.com/QL/ql-support/ql-training/>
- [26] "Variant analysis of web audio callback vulnerabilities in chrome," 2021. [Online]. Available: https://securitylab.github.com/research/chrome_task_queue_uaf/
- [27] "Website," 2023. [Online]. Available: <https://sites.google.com/view/qsast-survey/>
- [28] "CodeQL and the static tools logo test," 2021. [Online]. Available: <https://docs.microsoft.com/en-us/windows-hardware/drivers/devtest/static-tools-and-codeql>
- [29] "Microsoft open sources CodeQL queries used to hunt for Solorigate activity," 2021. [Online]. Available: shorturl.at/nyAX0
- [30] "Put in one bug and pop out more: An effective way of bug hunting in Chrome," in *Proc. Blackhat*, 2021, pp. 19–32.
- [31] "Using GitHub code scanning and CodeQL to detect traces of solorigate and other backdoors," 2021. [Online]. Available: shorturl.at/eDMQU
- [32] "AddressSanitizer check failed," 2022. [Online]. Available: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=101476
- [33] "CodeQL: CWE coverage for C and C++," 2022. [Online]. Available: <https://codeql.github.com/codeql-query-help/cpp-cwe/>
- [34] "SonarQube: CWE coverage for C and C++," 2022. [Online]. Available: <https://rules.sonarsource.com/cpp/tag/cwe/>
- [35] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating sequences from structured representations of code," 2018, *arXiv:1808.01400*.
- [36] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "Code2Vec: Learning distributed representations of code," in *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 1–29, Jan. 2019.
- [37] B. Aloraini, "Towards better static analysis security testing methodologies," 2020. [Online]. Available: <https://uwspace.uwaterloo.ca/handle/10012/16359>
- [38] B. Aloraini, M. Nagappan, D. M. German, S. Hayashi, and Y. Higo, "An empirical study of security warnings from static application security testing tools," *J. Syst. Softw.*, vol. 158, 2019, Art. no. 110427.
- [39] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshvyanyk, and A. Nadkarni, "Why crypto-detectors fail: A systematic evaluation of cryptographic misuse detection techniques," 2021, *arXiv:2107.07065*.
- [40] A. S. Ami, K. Kafle, K. Moran, A. Nadkarni, and D. Poshvyanyk, "Systematic mutation-based evaluation of the soundness of security-focused android static analysis techniques," *ACM Trans. Privacy Secur.*, vol. 24, no. 3, pp. 1–37, 2021.
- [41] A. Arusoae, S. Ciobăca, V. Craciun, D. Gavrilut, and D. Lucanu, "A comparison of open-source static analysis tools for vulnerability detection in C/C++ code," in *Proc. 19th Int. Symp. Symbolic Numeric Algorithms Sci. Comput.*, 2017, pp. 161–168.
- [42] M. Associates, "C and C++ source code analysis tools (open source and commercial)," 2021. [Online]. Available: <https://www.codeanalysistools.com/?cplplusplus>
- [43] P. Avgustinov, O. De Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in *Proc. 30th Eur. Conf. Object-Oriented Programm.*, 2016, pp. 2:1–2:25.
- [44] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Trans. Dependable Secure Comput.*, vol. 1, no. 1, pp. 11–33, First Quarter, 2004.
- [45] T. Ben-Nun, A. S. Jakobovits, and T. Hoefler, "Neural code comprehension: A learnable representation of code semantics," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 3589–3601.
- [46] A. Bessey et al., "A few billion lines of code later: Using static analysis to find bugs in the real world," *Commun. ACM*, vol. 53, no. 2, pp. 66–75, 2010.
- [47] D. Beyer, "Advances in automatic software verification: SV-COMP 2020," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst.*, 2020, pp. 347–367.
- [48] R. Bonett, K. Kafle, K. Moran, A. Nadkarni, and D. Poshvyanyk, "Discovering flaws in {Security-Focused} static analysis tools for android using systematic mutation," in *Proc. USENIX Secur.*, 2018, pp. 1263–1280.
- [49] G. Brat, J. A. Navas, N. Shi, and A. Venet, "IKOS: A framework for static analysis based on abstract interpretation," in *Proc. Int. Conf. Softw. Eng. Formal Methods*, 2014, pp. 271–277.
- [50] A. D. Brucker and S. SE, "Introducing security testing to developers," 2017. [Online]. Available: <https://brucker.ch/bibliography/download/2017/talk-brucker-cx-security-testing-2017.pdf>
- [51] J. Bundt, A. Fasano, B. Dolan-Gavitt, W. Robertson, and T. Leek, "Evaluating synthetic bugs," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2021, pp. 716–730.
- [52] G. Chatzieftheriou and P. Katsaros, "Test-driving static analysis tools in search of C code vulnerabilities," in *Proc. IEEE 35th Annu. Comput. Softw. Appl. Conf. Workshops*, 2011, pp. 96–103.
- [53] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," 2020, *arXiv:2002.12543*.
- [54] A. Chou, "Covery talk," 2008. [Online]. Available: https://www.usenix.org/legacy/event/sec08/tech/slides/engler_slides.pdf
- [55] Y. W. Chow, M. Schäfer, and M. Pradel, "Beware of the unexpected: Bimodal taint analysis," 2023, *arXiv:2301.10545*.
- [56] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2016, pp. 332–343.

- [57] Clang, “Clang static analyzer,” 2021. [Online]. Available: <https://clang-analyzer.lvm.org/>
- [58] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in *Proc. Conf. Rec. 4th ACM Symp. Princ. Programm. Lang.*, 1977, pp. 238–252.
- [59] R. Croft, D. Newlands, Z. Chen, and M. A. Babar, “An empirical study of rule-based and learning-based approaches for static application security testing,” in *Proc. 15th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas.*, 2021, pp. 1–12.
- [60] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, “Frama-C,” in *Proc. 10th Int. Conf. Softw. Eng. Formal Methods*, 2012, pp. 233–247.
- [61] cvedetails, “Browse vulnerabilities by date,” 2021. [Online]. Available: <https://www.cvedetails.com/browse-by-date.php>
- [62] A. Danial, “CLOC,” 2017. [Online]. Available: <https://github.com/AIDanial/cloc>
- [63] S. H. Ding, B. M. Fung, and P. Charland, “Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 472–489.
- [64] B. Dolan-Gavitt et al., “LAVA: Large-scale automated vulnerability addition,” in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 110–121.
- [65] J. Eberhardt, S. Steffen, V. Raychev, and M. Vechev, “Unsupervised learning of API aliasing specifications,” in *Proc. 40th ACM SIGPLAN Conf. Programm. Lang. Des. Implementation*, 2019, pp. 745–759.
- [66] Facebook, “Cpacheck,” 2021. [Online]. Available: <https://cpacheck.sourceforge.io/>
- [67] Facebook, “Facebook infer,” 2021. [Online]. Available: <https://fbinfer.com>
- [68] A. Fatima, S. Bibi, and R. Hanif, “Comparative study on static code analysis tools for C/C++,” in *Proc. 15th Int. Bhurban Conf. Appl. Sci. Technol.*, 2018, pp. 465–469.
- [69] M. Felderer, M. Büchler, M. Johns, A. D. Brucker, R. Breu, and A. Pretschner, “Security testing: A survey,” *Adv. Comput.*, vol. 101, pp. 1–51, 2016.
- [70] C. Gentsch, “Evaluation of open source static analysis security testing (SAST) tools for C,” DLR DW, Tech. Rep., Jan. 2020. [Online]. Available: <https://elib.dlr.de/133945/>
- [71] S. M. Ghaffarian and H. R. Shahriari, “Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey,” *ACM Comput. Surv.*, vol. 50, no. 4, pp. 56:1–56:36, 2017.
- [72] M. GitHub, “CodeQL,” 2021. [Online]. Available: <https://codeql.github.com/>
- [73] K. Goseva-Popstojanova and A. Perhinschi, “On the capability of static code analysis to detect security vulnerabilities,” *Inf. Softw. Technol.*, vol. 68, pp. 18–33, 2015.
- [74] D. Guido, “Trail of Bits’ adoption of Semgrep and CodeQL,” 2021. [Online]. Available: <https://twitter.com/dguido/status/1376581771916144646>
- [75] G. Hao et al., “Constructing benchmarks for supporting explainable evaluations of static application security testing tools,” in *Proc. Int. Symp. Theor. Aspects Softw. Eng.*, 2019, pp. 65–72.
- [76] R. T. Hartley, “Choosing static application testing tools,” Sandia National Lab.(SNL-NM), Albuquerque, NM, USA, Tech. Rep. SAND2019-5133C, 2019.
- [77] A. Hazimeh, A. Herrera, and M. Payer, “Magma: A ground-truth fuzzing benchmark,” in *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 4, no. 3, pp. 1–29, 2020.
- [78] J. Hua and H. Wang, “On the effectiveness of deep vulnerability detectors to simple stupid bug detection,” in *Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories*, 2021, pp. 530–534.
- [79] N. Imtiaz, B. Murphy, and L. Williams, “How do developers act on static analysis alerts? an empirical study of coverity usage,” in *Proc. 30th IEEE Int. Symp. Softw. Rel. Eng.*, 2019, pp. 323–333.
- [80] N. Imtiaz and L. Williams, “A synopsis of static analysis alerts on open source software,” in *Proc. 6th Annu. Symp. Hot Topics Sci. Secur.*, 2019, pp. 1–3.
- [81] D. Kang, E. Gan, P. Bailis, T. Hashimoto, and M. Zaharia, “Approximate selection with guarantees using proxies,” in *Proc. VLDB Endowment*, vol. 13, no. 11, pp. 1990–2003, 2020.
- [82] V. Kashyap et al., “Automated customized bug-benchmark generation,” in *Proc. 19th Int. Work. Conf. Source Code Anal. Manipulation*, 2019, pp. 103–114.
- [83] K. J. Kratkiewicz, “Evaluating static analysis tools for detecting buffer overflows in C code,” Harvard Univ., Cambridge MA, Tech. Rep. ADA511392, 2005.
- [84] “How the community powers github advanced security with codeql queries,” 2021. [Online]. Available: <https://shorturl.at/akqCU>
- [85] P. Larsen, A. Homescu, S. Brunthaler, and M. Franz, “SoK: Automated software diversity,” in *Proc. IEEE Symp. Secur. Privacy*, 2014, pp. 276–291.
- [86] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proc. 2nd IEEE/ACM Int. Symp. Code Gener. Optim.*, 2004, pp. 75–88.
- [87] S. Lee, H. Han, S. K. Cha, and S. Son, “Montage: A neural network language [model-guided] JavaScript engine fuzzer,” in *Proc. USENIX Secur. Symp.*, 2020, pp. 2613–2630.
- [88] A. Y. Levy, I. S. Mumick, and Y. Sagiv, “Query optimization by predicate move-around,” in *Proc. 20th Int. Conf. Very Large Data Bases*, 1994, pp. 96–107.
- [89] J. Li, “Vulnerabilities mapping based on OWASP-SANS: A survey for static application security testing (SAST),” *Ann. Emerg. Technol. Comput.*, vol. 4, no. 3, pp. 1–8, 2020.
- [90] X. Li, Q. Yu, and H. Yin, “PalmTree: Learning an assembly language model for instruction embedding,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3236–3251.
- [91] S. Lipp, S. Banescu, and A. Pretschner, “An empirical study on the effectiveness of static C code analyzers for vulnerability detection,” in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 544–555.
- [92] B. Livshits, “In defense of soundness: A manifesto,” *Commun. ACM*, vol. 58, no. 2, pp. 44–46, 2015.
- [93] B. Lu, W. Dong, L. Yin, and L. Zhang, “Evaluating and integrating diverse bug finders for effective program analysis,” in *Proc. Int. Conf. Softw. Anal., Testing, Evol.*, 2018, pp. 51–67.
- [94] A. Machiry, N. Redini, E. Gustafson, H. Aghakhani, C. Kruegel, and G. Vigna, “Towards automatically generating a sound and complete dataset for evaluating static analysis tools,” in *Proc. Workshop Binary Anal. Res.*, 2019. [Online]. Available: <https://par.nsf.gov/biblio/10155111-towards-automatically-generating-sound-complete-dataset-evaluating-static-analysis-tools>
- [95] R. Mahmood and Q. H. Mahmoud, “Evaluation of static analysis tools for finding vulnerabilities in java and C/C++ source code,” 2018, *arXiv:1805.09040*.
- [96] A. Mantovani, L. Compagna, Y. Shoshitaishvili, and D. Balzarotti, “The convergence of source code and binary vulnerability discovery—a case study,” in *Proc. Asia Conf. Comput. Commun. Secur.*, 2022, pp. 602–615.
- [97] M. Martinez, J. C. Ruiz, N. Antunes, D. De Andres, and M. Vieira, “A multi-criteria analysis of benchmark results with expert support for security tools,” *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2151–2164, Jul./Aug. 2020.
- [98] R. K. McLean, “Comparing static security analysis tools using open source software,” in *Proc. IEEE 6th Int. Conf. Softw. Secur. Rel. Companion*, 2012, pp. 68–74.
- [99] F. Meade, “Cas static analysis tool study-methodology,” 2012. [Online]. Available: <https://www.nist.gov/document/samate-document-cas-static-analysis-tool-study-methodology-december-2012>
- [100] J. Metzman, L. Szekeres, L. Simon, R. Sprabery, and A. Arya, “FuzzBench: An open fuzzer benchmarking platform and service,” in *Proc. 29th ACM Joint Eur. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1393–1403.
- [101] mitre, “Cwe view: Weaknesses in C++,” 2022. [Online]. Available: <https://cwe.mitre.org/data/definitions/659.html>
- [102] J. Moerman, S. Smetsers, and M. Schoolderman, “Evaluating the performance of open source static analysis tools,” Bachelor Thesis, Dept. Sci., Radboud Univ., The Netherlands, vol. 24, 2018.
- [103] D. Mu et al., “Understanding the reproducibility of crowd-reported security vulnerabilities,” in *Proc. USENIX Secur.*, 2018, pp. 919–936.
- [104] NASA, “ikos,” 2021. [Online]. Available: <https://github.com/NASA-SW-VnV/ikos>
- [105] NIST, “Juliet test suite for C/C++,” 2021. [Online]. Available: <https://samate.nist.gov/SRD/testsuite.php>
- [106] NIST, “NIST samate,” 2022. [Online]. Available: <https://www.nist.gov/itl/ssd/software-quality-group/samate/introduction-samate>
- [107] P. Nunes, I. Medeiros, J. C. Fonseca, N. Neves, M. Correia, and M. Vieira, “Benchmarking static analysis tools for web security,” *IEEE Trans. Rel.*, vol. 67, no. 3, pp. 1159–1175, Sep. 2018.

- [108] R. M. Parizi, K. Qian, H. Shahriar, F. Wu, and L. Tao, "Benchmark requirements for assessing software security vulnerability testing tools," in *Proc. IEEE 42nd Ann. Comput. Softw. Appl. Conf.*, 2018, pp. 825–826.
- [109] I. Pashchenko, S. Dashevskyi, and F. Massacci, "Delta-bench: Differential benchmark for static analysis security testing tools," in *Proc. ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas.*, 2017, pp. 163–168.
- [110] J. Patra and M. Pradel, "Semantic bug seeding: A learning-based approach for creating realistic bugs," in *Proc. 29th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 906–918.
- [111] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "TREX: Learning execution semantics from micro-traces for binary similarity," 2020, *arXiv:2012.08680*.
- [112] A. Plaskett, "CVE-2020-9967 - Apple macOS 6LowPAN Vulnerability," 2020. [Online]. Available: <https://alexplaskett.github.io/CVE-2020--9967/>
- [113] R2C, "Semgrep," 2021. [Online]. Available: <https://semgrep.dev>
- [114] R. N. Rajapakse, M. Zahedi, and M. A. Babar, "An empirical analysis of practitioners' perspectives on security tool integration into devops," in *Proc. ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas.*, 2021, pp. 1–12.
- [115] S. Roy, A. Pandey, B. Dolan-Gavitt, and Y. Hu, "Bug synthesis: Challenging bug-finding tools with deep faults," in *Proc. ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2018, pp. 224–234.
- [116] N. Rutar, C. B. Almazan, and J. S. Foster, "A comparison of bug finding tools for java," in *Proc. 15th Int. Symp. Softw. Rel. Eng.*, 2004, pp. 245–256.
- [117] W. F. Santoso and D. S. S. Sahid, "Implementation and performance analysis development security operations (DevSecOps) using static analysis and security testing (SAST)," in *Proc. Int. Appl. Bus. Eng. Conf.*, 2021, pp. 17–19.
- [118] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 805–824, Sep. 2016.
- [119] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 28–28.
- [120] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA, 2021, pp. 968–980.
- [121] Shiftleft, "How shiftleft achieved the highest SAST score on the OWASP benchmark," 2021. [Online]. Available: shorturl.at/rJZ06
- [122] S. Shiraishi, V. Mohan, and H. Marimuthu, "Test suites for benchmarks of static analysis tools," in *Proc. Int. Symp. Softw. Rel. Eng. Workshops*, 2015.
- [123] SonarSource, "Sonarqube," 2021. [Online]. Available: <https://www.sonarqube.org/>
- [124] M. Stamp, *Information Security: Principles and Practice*. Hoboken, NJ, USA: Wiley, 2011.
- [125] Y. Sui and J. Xue, "SVF: Interprocedural static value-flow analysis in LLVM," in *Proc. 25th Int. Conf. Compiler Construction*, 2016, pp. 265–266.
- [126] Synopsys, "Coverity," 2022. [Online]. Available: <https://scan.coverity.com/>
- [127] Synopsys, "Coverity: CWE support," 2022. [Online]. Available: <https://web.archive.org/web/20220428092203/https://www.synopsys.com/software-integrity/security-testing/static-analysis-sast/coverity-cwe.html>
- [128] T. J. Team, "Joern," 2021. [Online]. Available: <https://joern.io>
- [129] F. Thung et al., "To what extent could we detect field defects? an empirical study of false negatives in static bug finding tools," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2012, pp. 50–59.
- [130] A. Tools, "Awesome static analysis," 2021. [Online]. Available: <https://github.com/analysis-tools-dev/static-analysis>
- [131] J. Vadayath et al., "Arbiter: Bridging the static and dynamic divide in vulnerability discovery on binary programs," in *Proc. USENIX Secur.*, 2022, pp. 413–430.
- [132] S. Wagner, F. Deissenboeck, M. Aichner, J. Wimmer, and M. Schwalb, "An evaluation of two bug pattern tools for java," in *Proc. 1st Int. Conf. Softw. Testing, Verification, Validation*, 2008, pp. 248–257.
- [133] S. Wagner, J. Jürjens, C. Koller, and P. Trischberger, "Comparing bug finding tools with reviews and tests," in *Proc. 17th IFIP TC6/WG 6.1 Int. Conf. Testing Commun. Syst.*, 2005, pp. 40–55.
- [134] J. Wang, Y. Huang, S. Wang, and Q. Wang, "An approach to detecting bugs in pattern-based bug detectors," 2021, *arXiv:2109.02245*.
- [135] WhiteSource, "Mend bolt," 2022. [Online]. Available: <https://www.mend.io/free-developer-tools/bolt>
- [136] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.
- [137] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proc. IEEE Symp. Secur. Privacy*, 2014, pp. 590–604.
- [138] J. Yang, L. Tan, J. Peyton, and K. A. Duer, "Towards better utilizing static application security testing," in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2019, pp. 51–60.
- [139] H. Yu, S. Zekun, and B. Dolan-Gavitt, "Characterizing and improving bug-finders with synthetic bugs," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering*, 2022, pp. 971–982.
- [140] M. Zalewski, "American fuzzy lop (AFL)," 2021. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [141] G. P. Zero, "Weggli," 2021. [Online]. Available: <https://github.com/googleprojectzero/weggli>



Zongjie Li (Graduate Student Member, IEEE) is currently working toward the PhD degree with the Department of Computer Science and Engineering at the Hong Kong University of Science and Technology, Hong Kong SAR. His research interests include software engineering, large language model security, and static analysis.



Zhibo Liu is a currently working toward the PhD degree with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong SAR. His research interests include reverse engineering, software engineering, and computer security.



Wai Kin Wong is currently working toward the PhD degree with the Department of Computer Science and Engineering, HKUST, Hong Kong SAR. His research interests include software security and reverse engineering.



Pingchuan Ma is currently working toward the PhD degree with the Department of Computer Science and Engineering, HKUST, Hong Kong SAR. His research interests include software engineering and data management.



Shuai Wang (Member, IEEE) received the PhD degree in informatics from Penn State University, USA, in 2018. He is an Assistant Professor with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong SAR. His research interests include cybersecurity and software engineering.